Topic 5B
PUNEH Processor

Outline

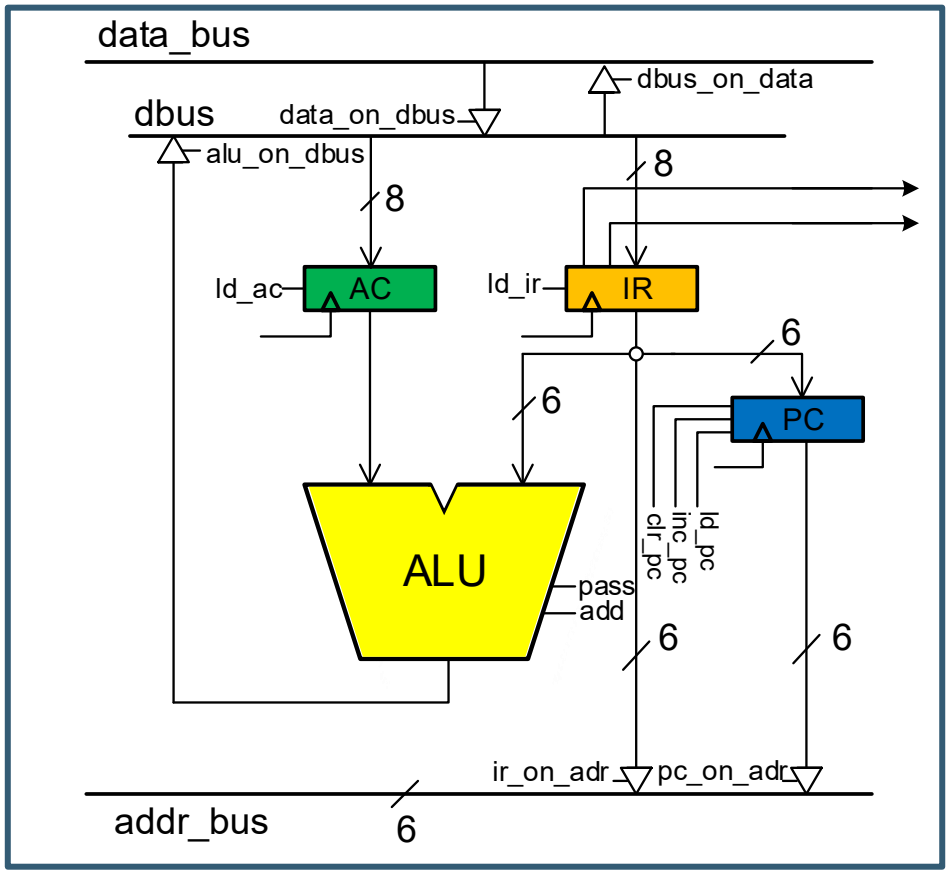
- Computer Hardware
- Computer Software
- Instruction Set Architecture
- PUNEH as a Test-Case Processor

Outline

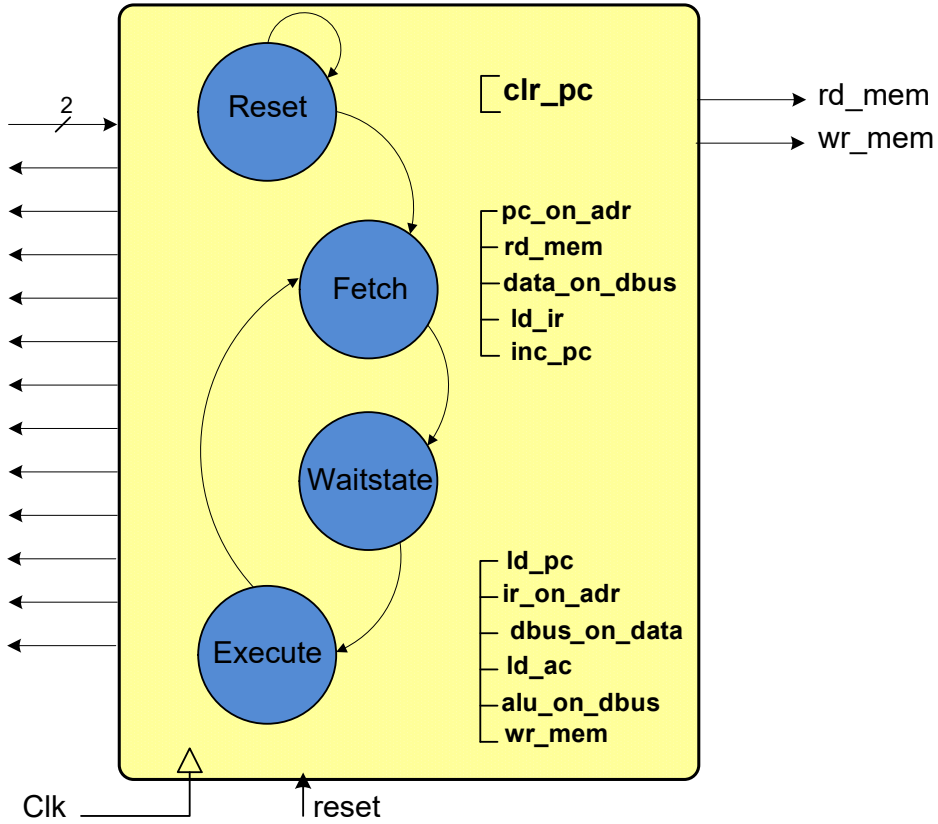
- *Computer Hardware*
- Computer Software
- Instruction Set Architecture
- PUNEH as a Test-Case Processor

Computer Hardware

- **RTL Design**
  - **Datapath**
  - **Controller**



opcode
ir_on_adr
pc_on_adr
dbus_on_data
data_on_bus
ld_ir
ld_ac
ld_pc
inc_pc
clr_pc
pass
alu_on_dbus
add

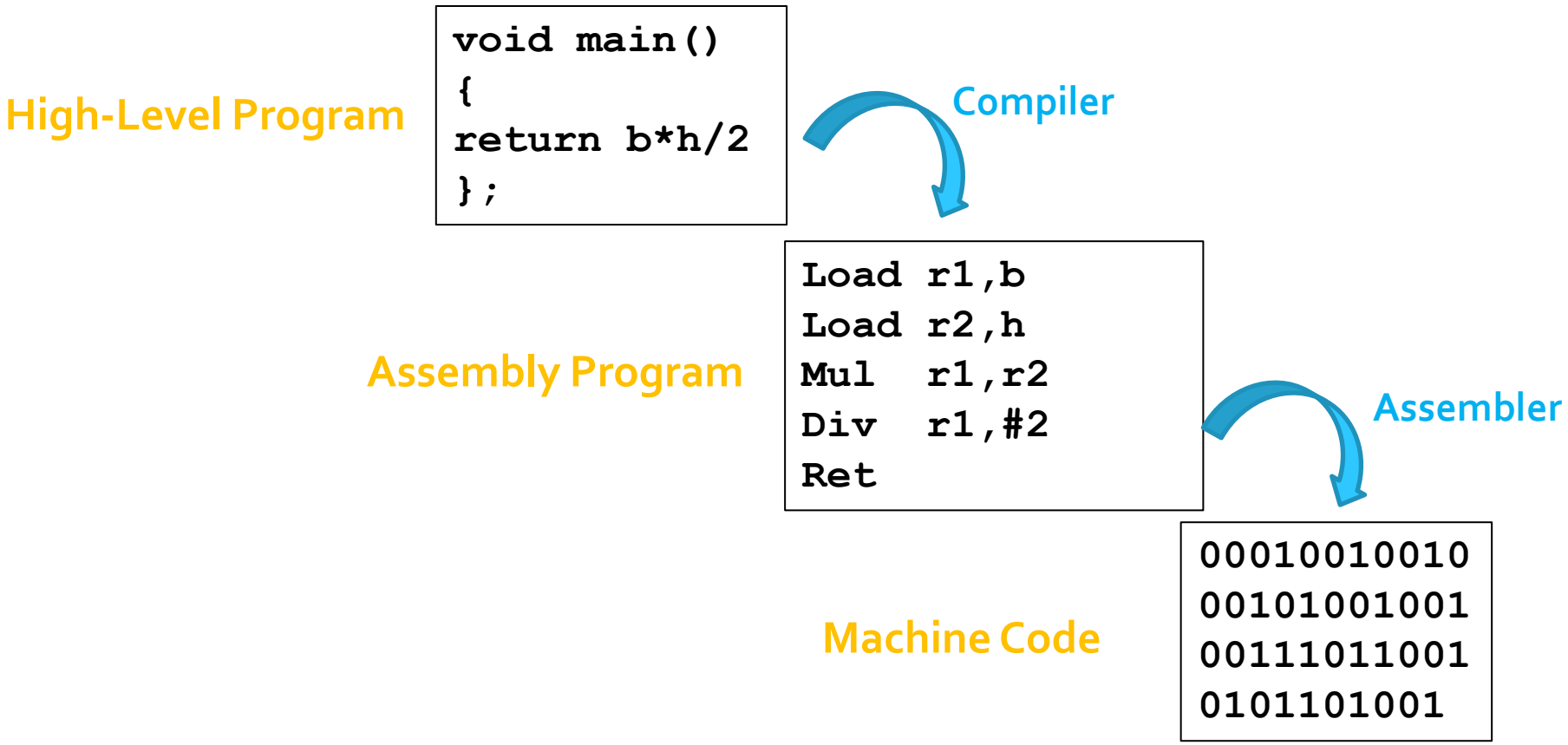


Outline

- Computer Hardware
- *Computer Software*
- Instruction Set Architecture
- PUNEH as a Test-Case Processor

Computer Software

- **Level of Programming Languages**



Computer Software

- **High-Level Language**
  - **Such as C, FORTRAN, or Pascal that enables a programmer to write programs that are more or less independent of a particular type of computer.**
  - **Such languages are considered high-level because they are closer to human languages and further from machine languages.**

```
void main()
{
return b*h/2
};
```

Computer Software

- **Assembly Language**
  - **Machine language programs are tedious and error-prone to write, and difficult to understand.**
  - **The programmers used a symbolic notation called assembly language.**
  - **Assembly language is easier to use than machine language.**

```
Load r1,b
Load r2,h
Mul  r1,r2
Div  r1,#2
Ret
```

Computer Software

- **Machine Languge**
  - Machine language is a collection of binary digits or bits
  - The computer alphabet is just two letters 0 and 1
  - Instructions are binary numbers meaningful for a computer
  - The binary notation is referred to as the machine language

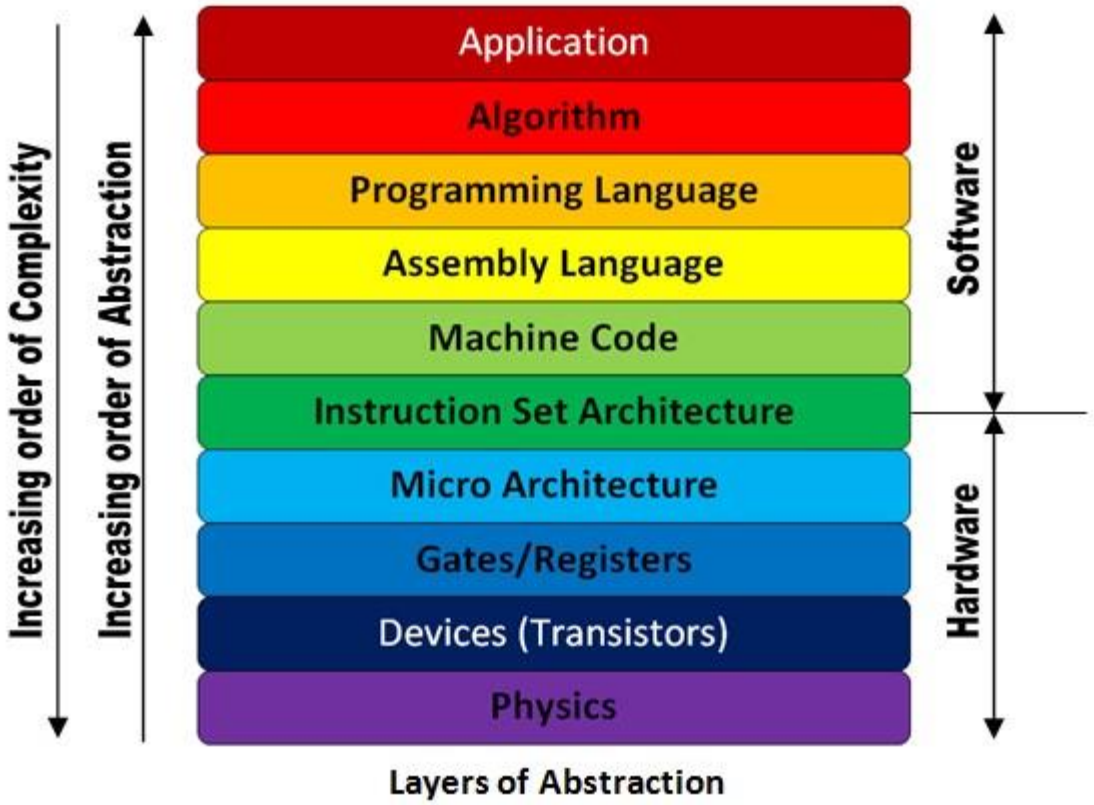
```
00010010010
00101001001
00111011001
0101101001
```

Outline

- Computer Hardware
- Computer Software
- *Instruction Set Architecture*
- PUNEH as a Test-Case Processor

Instruction Set Architecture

- Instruction set architecture (ISA) is an abstract model of a processor
- Hardware/Software Interface

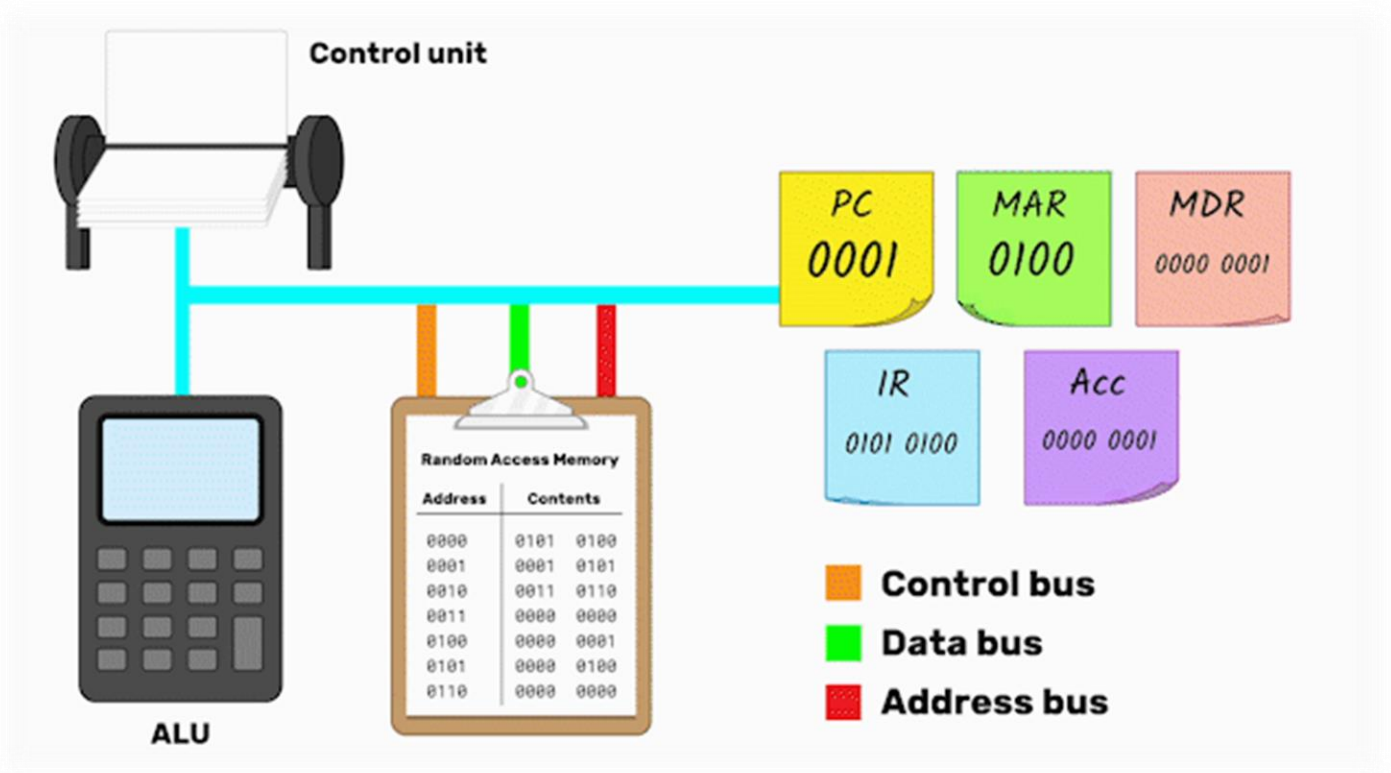


Outline

- Computer Hardware
- Computer Software
- Instruction Set Architecture
- *PUNEH as a Test-Case Processor*

PUNEH as a Test-Case Processor

- A processor is a digital circuit that performs tasks defined for it in its memory
- Tasks are broken into individual instructions



PUNEH as a Test-Case Processor

- **CPU Specification**

The CPU design begins with the specification of the CPU, including:

1. **Number of general-purpose registers**
2. **Memory organization**
   - **Whether byte or word addressable.**
3. **Instruction format**
4. **Addressing modes**
   - **direct, indirect, …**

PUNEH as a Test-Case Processor

- **PUNEH CPU Specification**
  (Processor that is Useful and New for Education of Hardware)

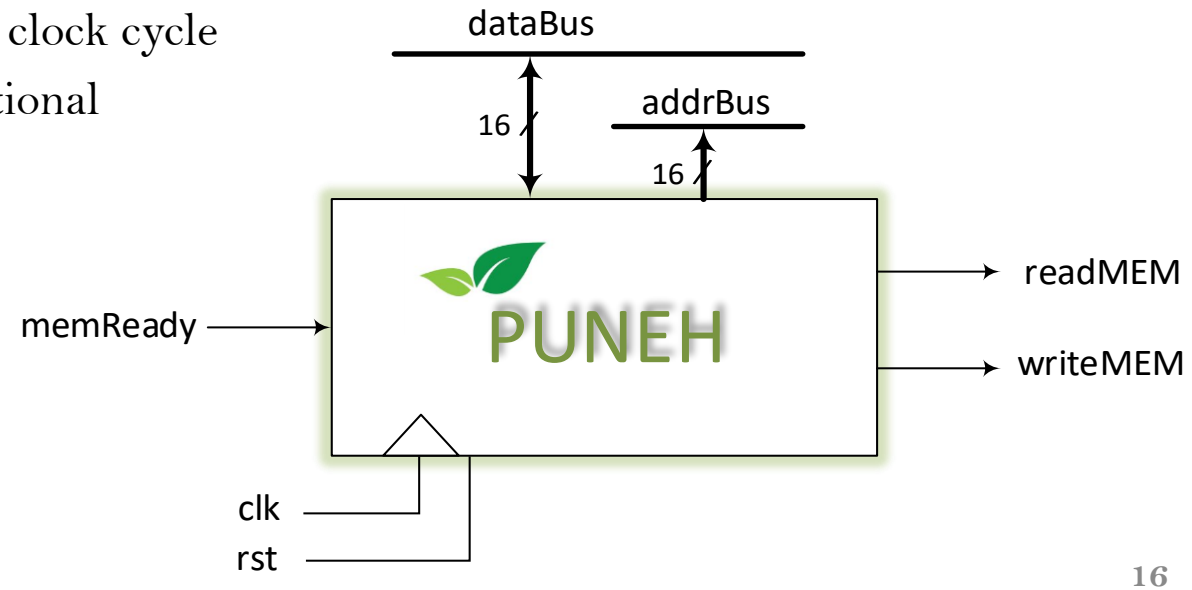
1. **Number of general-purpose registers**
   - PC (Program Counter)
   - OF (Offset)
   - AC (Accumulator)
   - IR (Instruction Register)

PUNEH as a Test-Case Processor

- **PUNEH CPU Specification**
  (Processor that is Useful and New for Education of Hardware)

2. Memory organization

- 16-bit address bus
- Capable of addressing 2^{16} locations of memory through its 16-bit address bus (addrbus)
- 16-bit (2 bytes) addressable
- **Write access** to the memory requires 1 clock cycle
- **Read access** to the memory is combinational

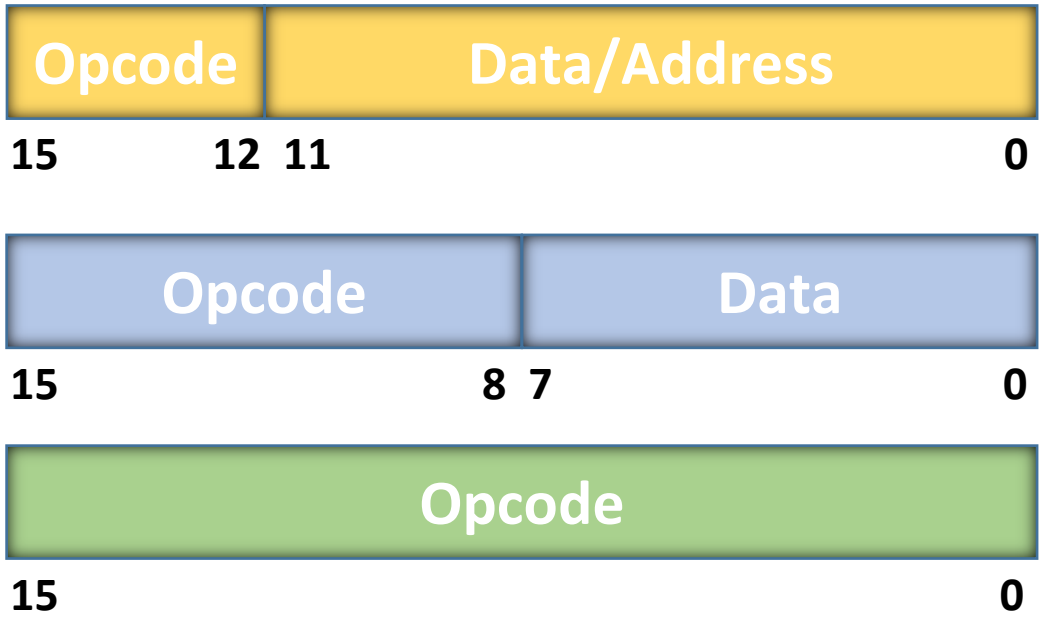


PUNEH as a Test-Case Processor

- **PUNEH CPU Specification**
  (Processor that is Useful and New for Education of Hardware)

3. Instruction format

- 16-bit instruction format



PUNEH as a Test-Case Processor

- **PUNEH CPU Specification**
  (Processor that is Useful and New for Education of Hardware)

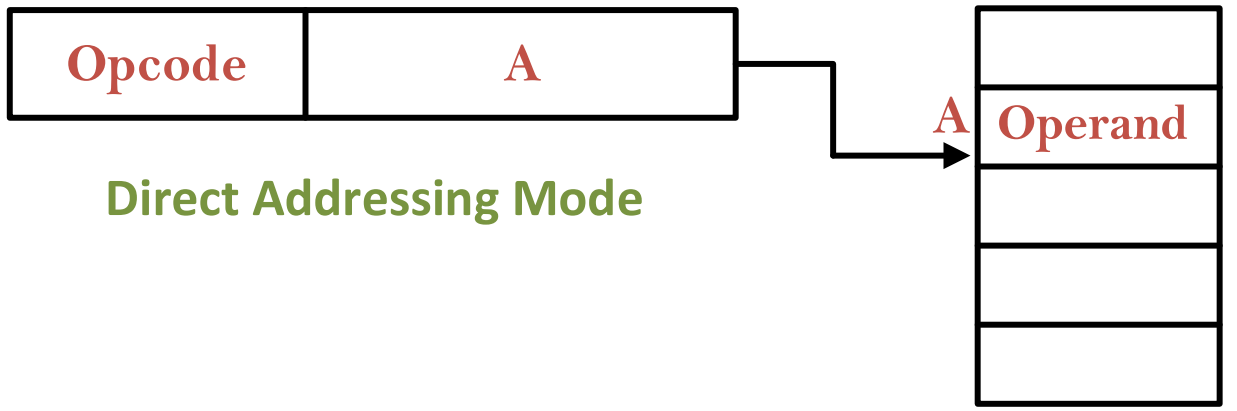
4. **Addressing modes**
   - ***Immediate:***
     - The operand is specified in the instruction explicitly.
     - Instead of address field, an operand field is present that contains the operand.
   - Direct
   - Indirect

Immediate Addressing Mode

PUNEH as a Test-Case Processor

- **PUNEH CPU Specification**
  (Processor that is Useful and New for Education of Hardware)

4. **Addressing modes**
   - Immediate:
   - *Direct*
     - The address field of the instruction contains the effective address of the operand.
     - Only one reference to memory is required to fetch the operand.
   - Indirect

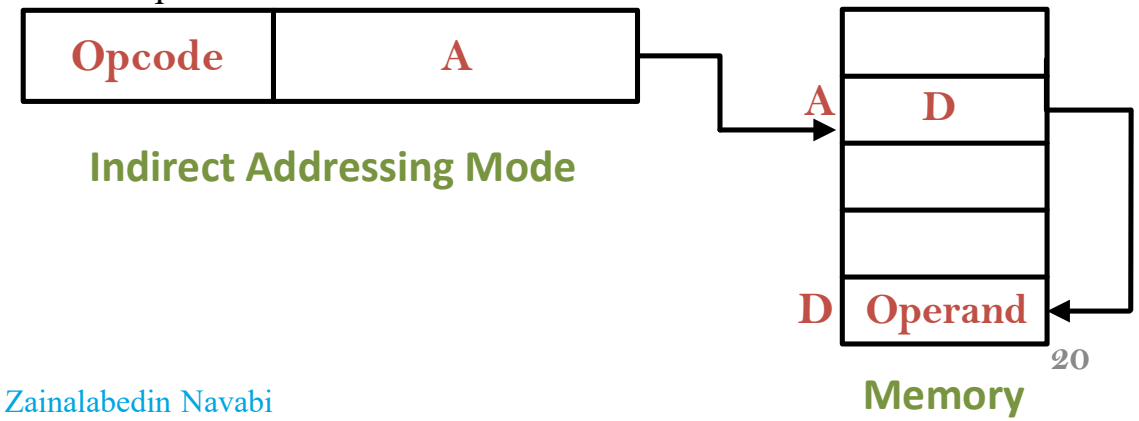


Direct Addressing Mode

PUNEH as a Test-Case Processor

- **PUNEH CPU Specification**
  (Processor that is Useful and New for Education of Hardware)

4. **Addressing modes**
   - Immediate:
   - Direct
   - ***Indirect***
     - The address field of the instruction specifies the address of memory location that contains the effective address of the operand.
     - Two references to memory are required to fetch the operand.



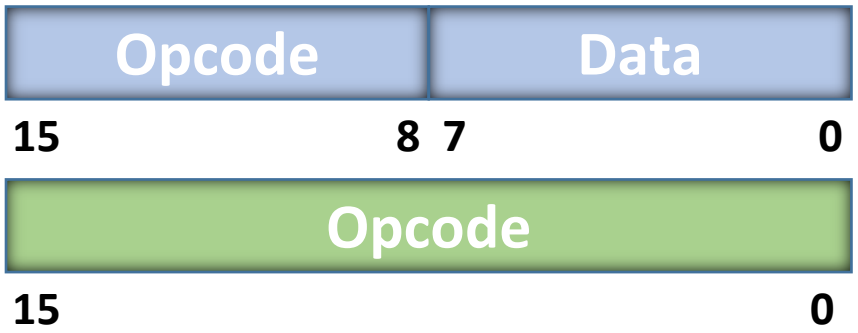
PUNEH as a Test-Case Processor

- **PUNEH has a total of 26 instructions**
- **Three addressing modes for address**
  - X_m indicating immediate addressing mode
  - X_a indicating direct addressing mode
  - X_n indicating indirect addressing mode
- **General Purpose Instructions**
  - **Data Transfer Instructions**
  - **Logical Instructions**
  - **Arithmetic Instructions**
  - **Shift Instructions**
  - **Control Transfer Instructions**
  - **Flag Control Instructions**

Instruction Type	Opcode	Instruction	Description
Full Address Instr.	0000	LDm	Load Accumulator Immediate
	0001	LDa	Load Accumulator Addressed
	0010	LDn	Load Accumulator Indirect
	0011	STa	Store Accumulator Addressed
	0100	STn	Store Accumulator Indirect
	0101	INa	Increment memory Address
	0110	ANm	AND Accumulator Immediate
	0111	ANa	AND Accumulator Addressed
	1000	ADm	Add Accumulator Immediate
	1001	ADa	Add Accumulator Addressed
	1010	ADn	Add Accumulator Indirect
	1011	MLa	Multiply Accumulator Addressed
	1100	JMa	Jump Addressed (unconditional)
	1101	JMn	Jump Indirect
	1110	JSR	Jump Sub-routine

PUNEH as a Test-Case Processor

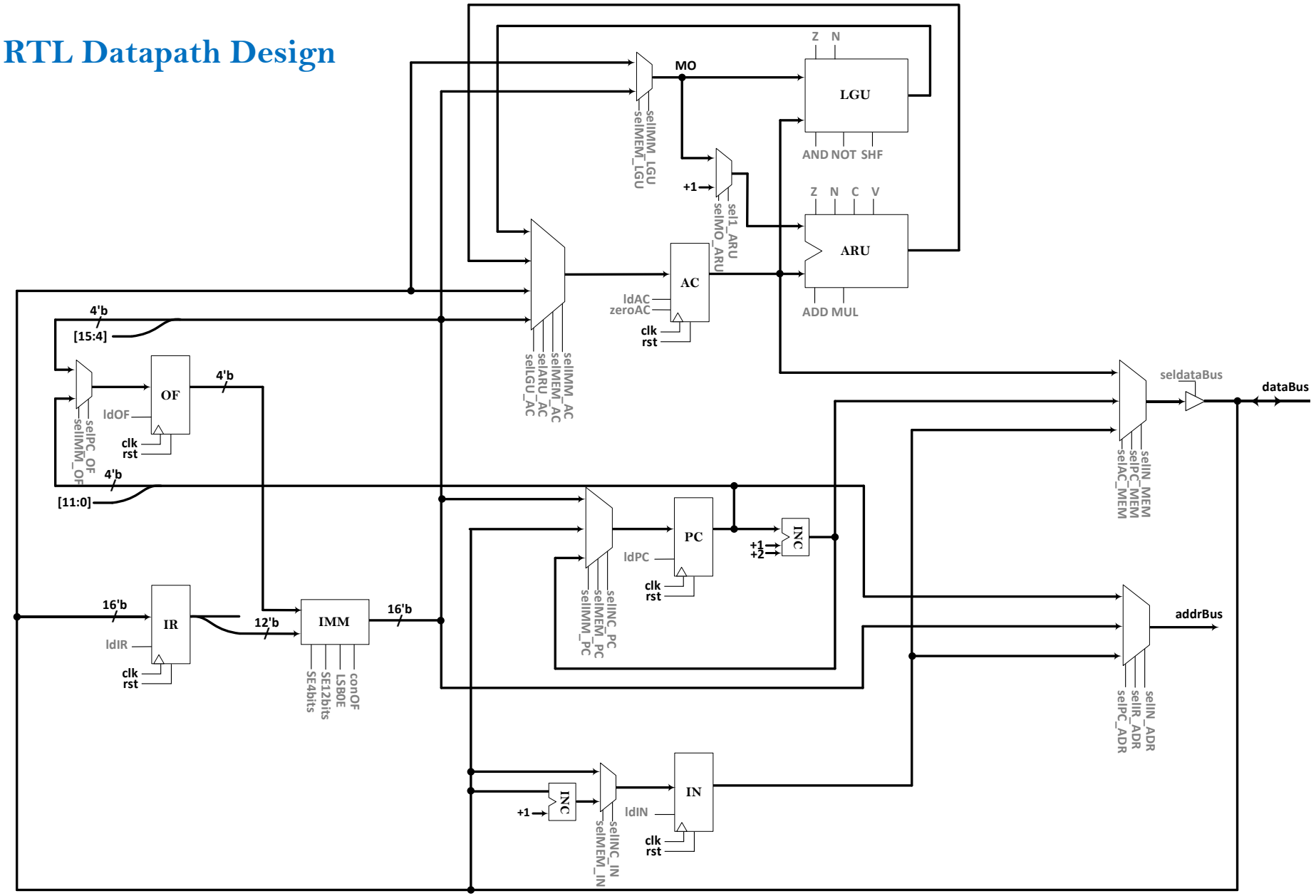
- General Purpose Instructions
  - Data Transfer Instructions
  - Logical Instructions
  - Arithmetic Instructions
  - Shift Instructions
  - Control Transfer Instructions
  - Flag Control Instructions

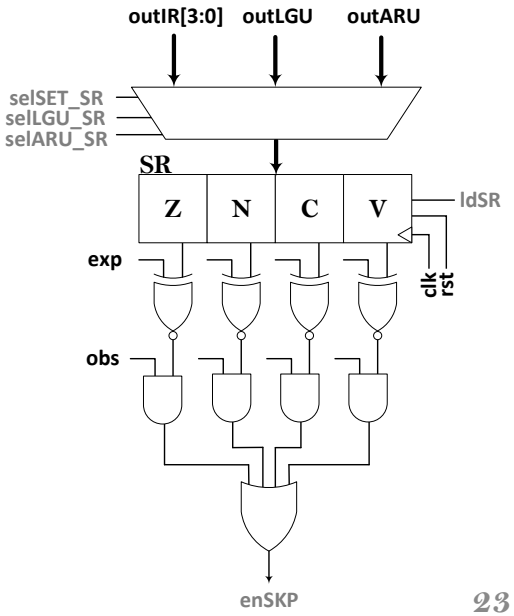


Instruction Type	Opcode		Instruction	Description
No Address Instr.	1111	0000 00000000	LPO	Load PC to Offset register
		0000 00000001	LOP	Load Offset register to PC
		0000 00000010	ACZ	Accumulator Zero
		0000 00000011	ACN	Accumulator NOT
		0000 00000100	ACI	Accumulator Increment
		0001	LOm	Load Offset register Immediate
		0010	SRA	Shift Right Arithmetic
		0011	SRL	Shift Right Logical
		0100	SLL	Shift Left Logical
		0101	$SKP_{Z,N,C,V}$	Skip Zero, Negative, Carry, Overflow (obs, exp)
		0110	$SET_{Z,N,C,V}$	Set Zero, Negative, Carry, Overflow (enb, val)

PUNEH as a Test-Case Processor

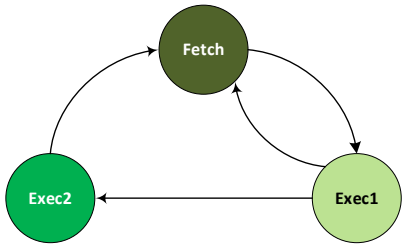
- **RTL Datapath Design**

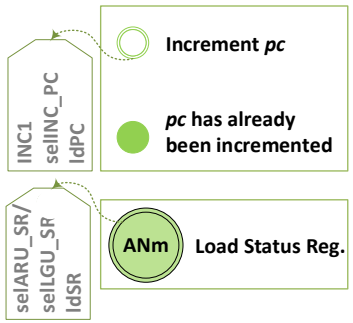


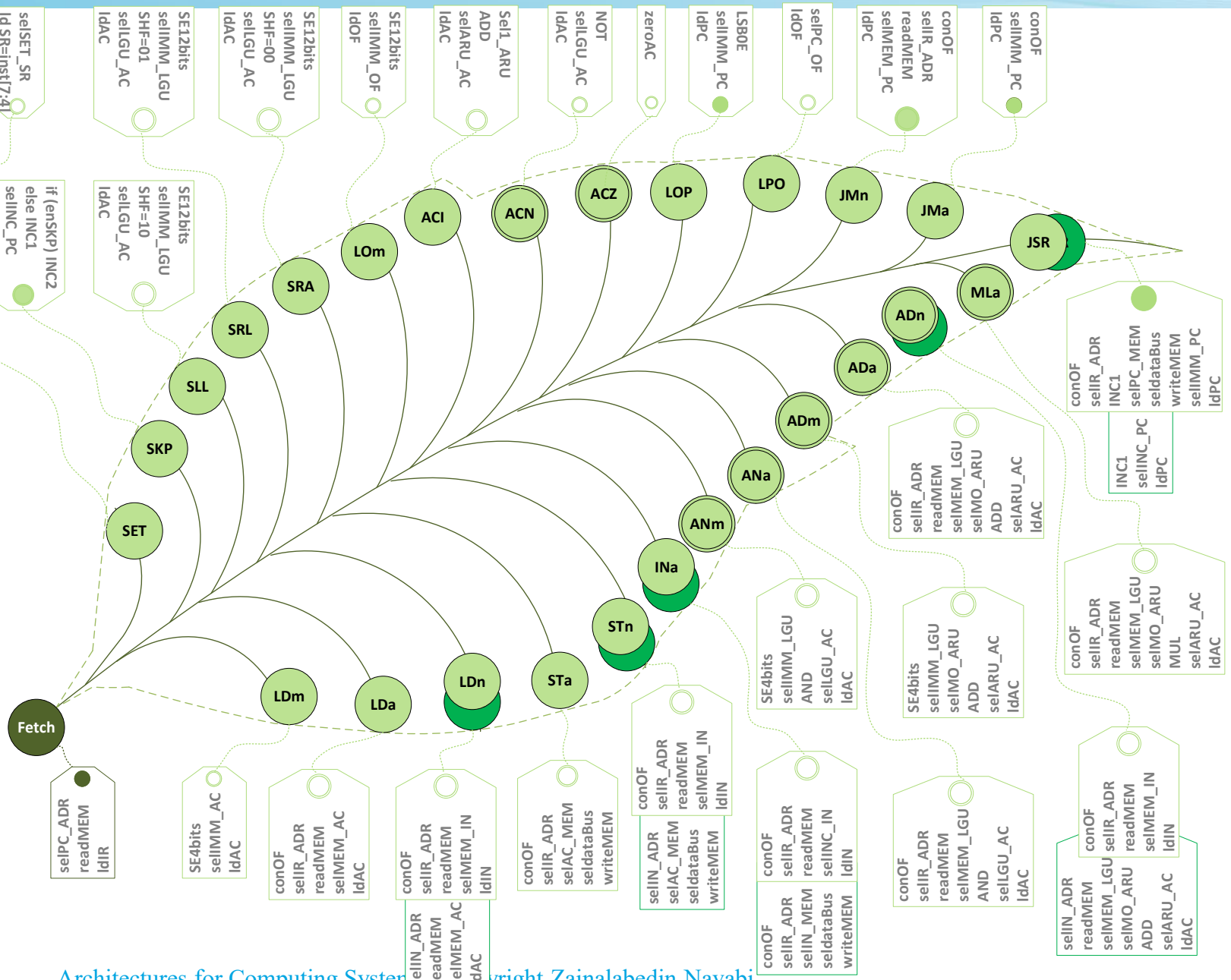


PUNEH as a Test-Case Processor

- **Controller Design**







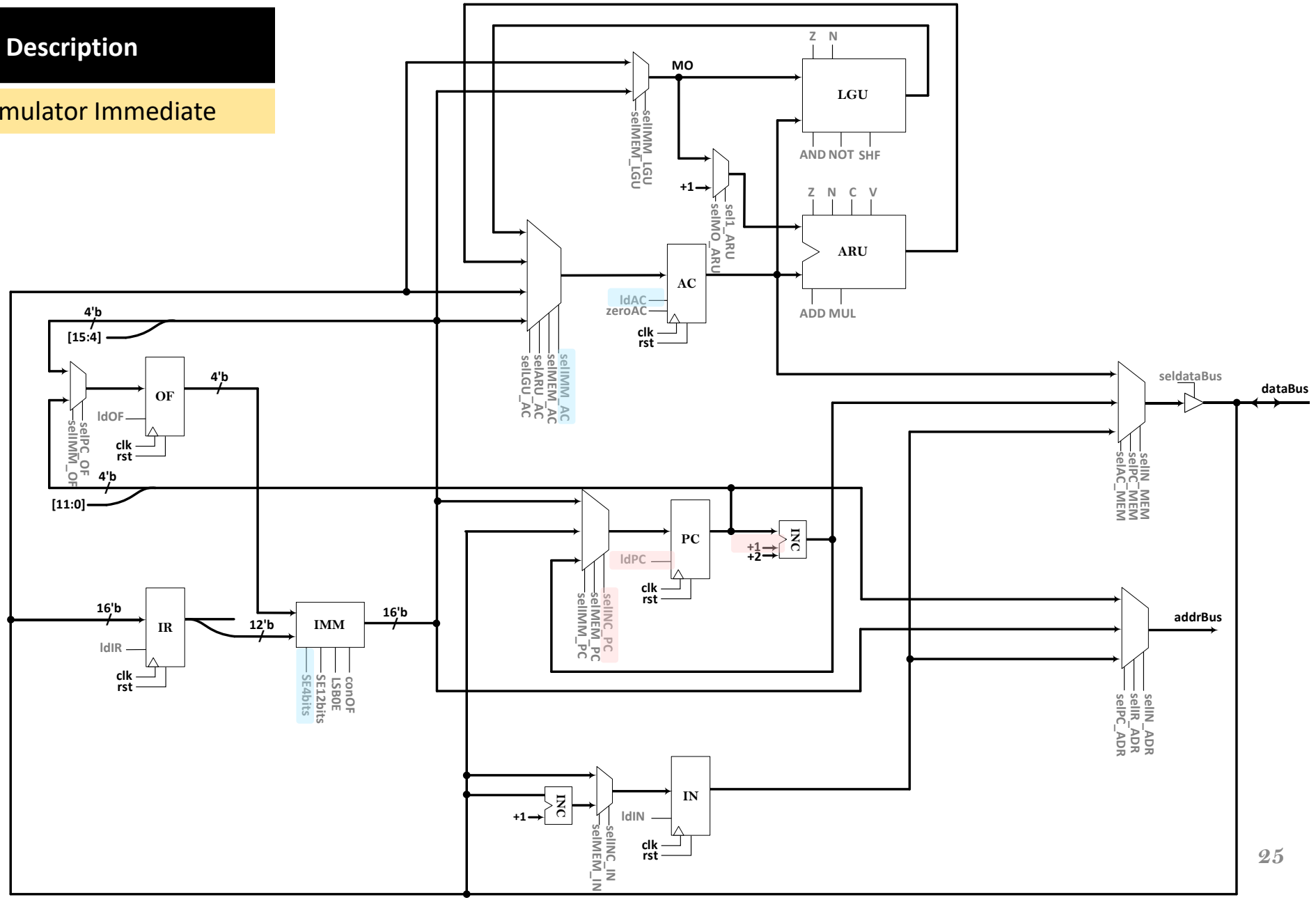
PUNEH as a Test-Case Processor

Opcode	Instruction	Description
0000	LDm	Load Accumulator Immediate

Exec1

LDm :

```
SE4bits = 1;
selIMM_AC = 1;
ldAC = 1;
INC1 = 1;
selINC_PC = 1;
ldPC = 1;
```

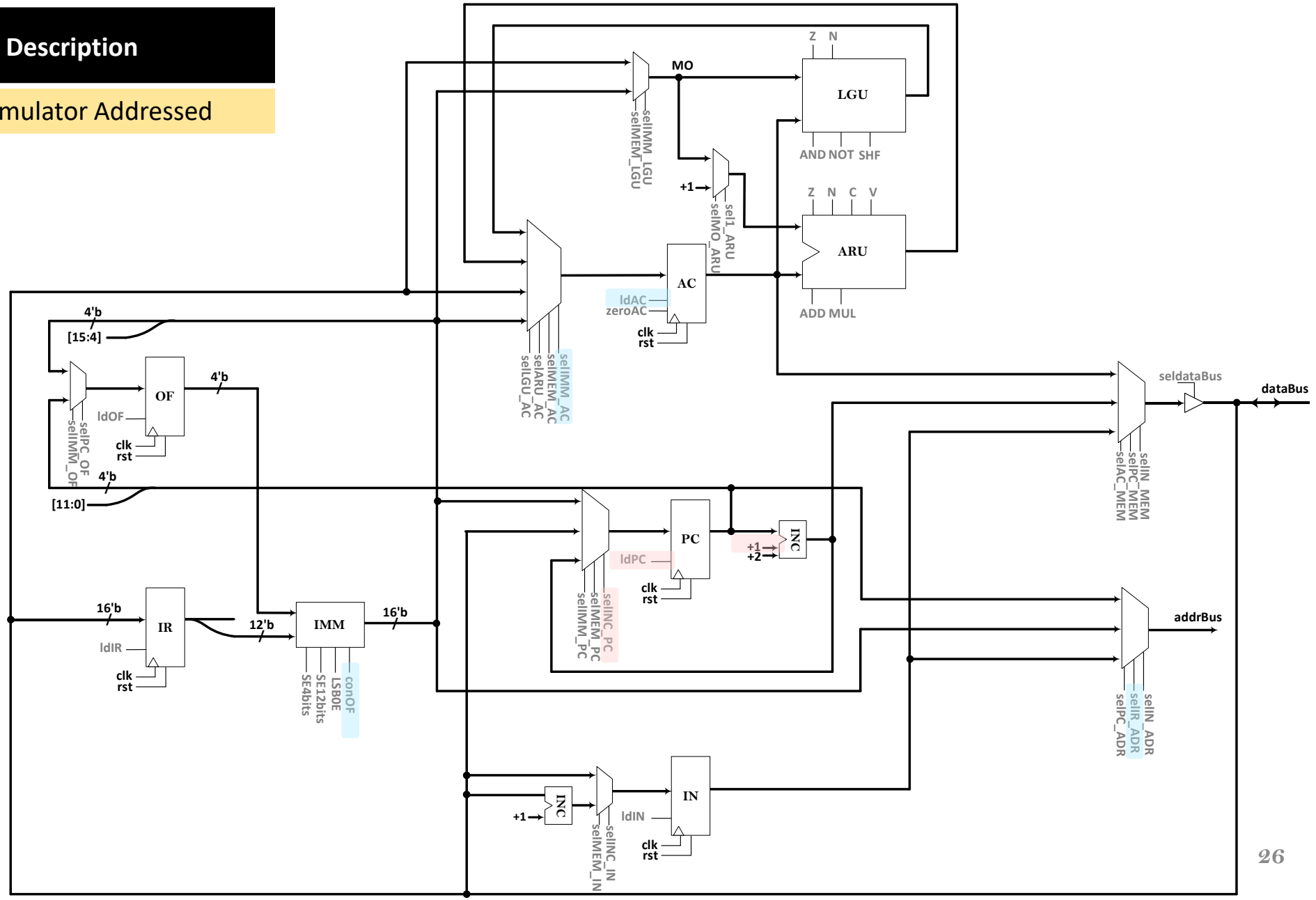


PUNEH as a Test-Case Processor

Opcode	Instruction	Description
0001	LDa	Load Accumulator Addressed

Exec1

```
LDa :
    conOF = 1;
    selIR_ADR = 1;
    readMEM = 1;
    selMEM_AC = 1;
    ldAC = 1;
    INC1 = 1;
    selINC_PC = 1;
    ldPC = 1;
```



PUNEH as a Test-Case Processor

Opcode	Instruction	Description
0010	LDn	Load Accumulator Indirect

Exec1

LDn :

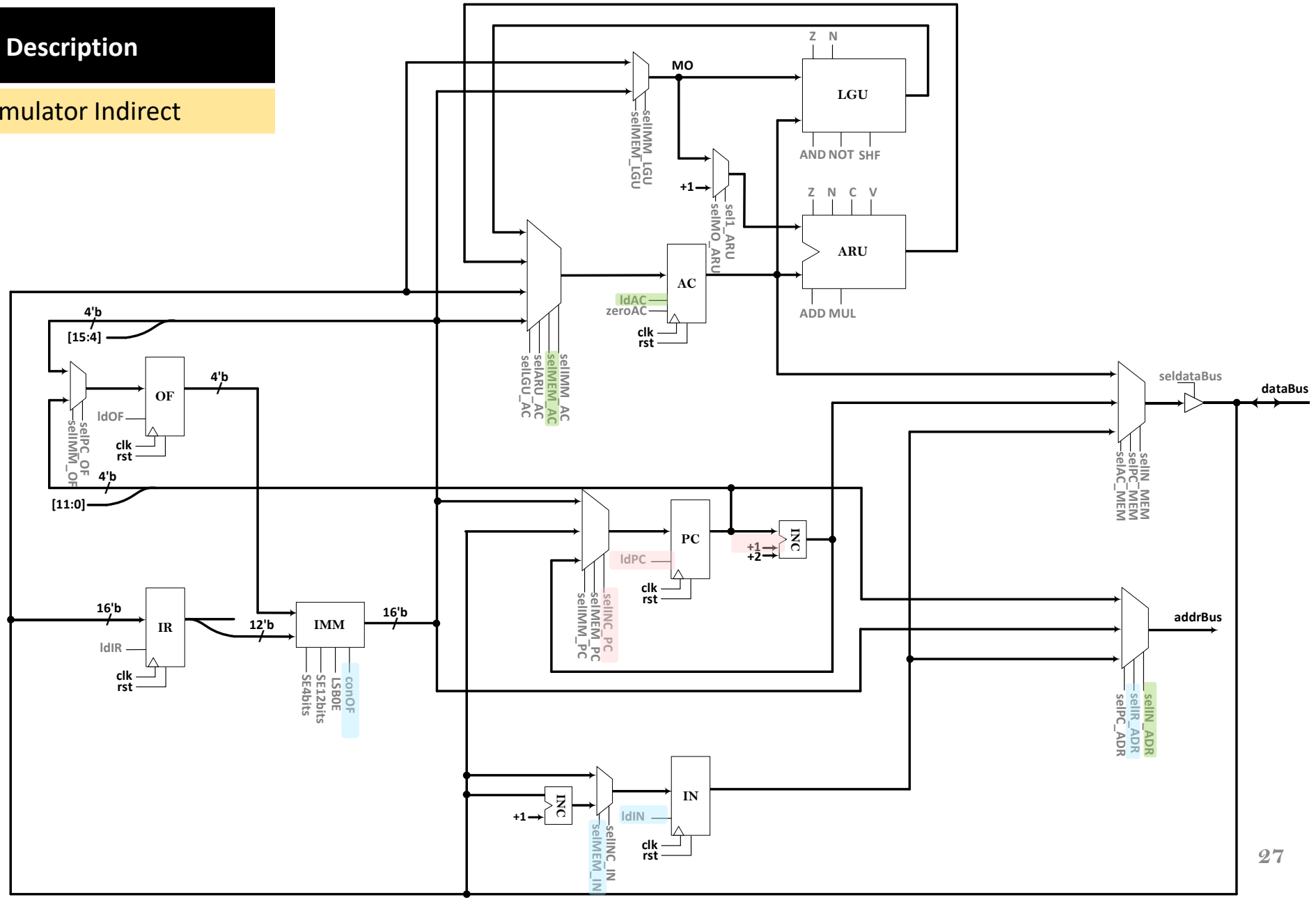
```
conOF = 1;
selIR_ADR = 1;
readMEM = 1;
selMEM_IN = 1;
ldIN = 1;
```

Exec2

LDn :

```
selIN_ADR = 1;
readMEM = 1;
selMEM_AC = 1;
ldAC = 1;
```

```
INC1 = 1;
selINC_PC = 1;
ldPC = 1;
```

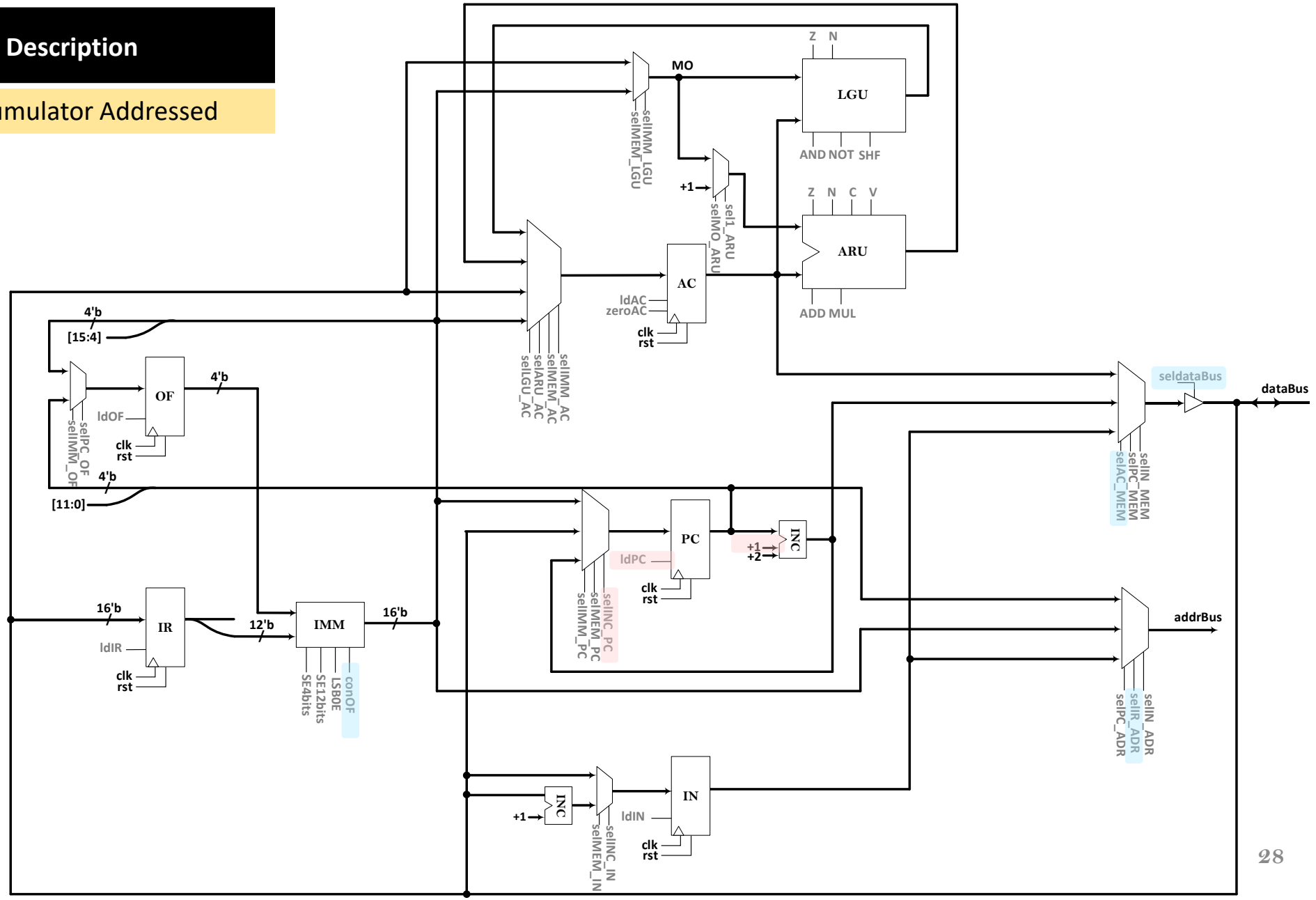


PUNEH as a Test-Case Processor

Opcode	Instruction	Description
0011	STa	Store Accumulator Addressed

Exec1

```
STa :
    conOF = 1;
    selIR_ADR = 1;
    selAC_MEM = 1;
    seldataBus = 1;
    writeMEM = 1;
    INC1 = 1;
    selINC_PC = 1;
    ldPC = 1;
```



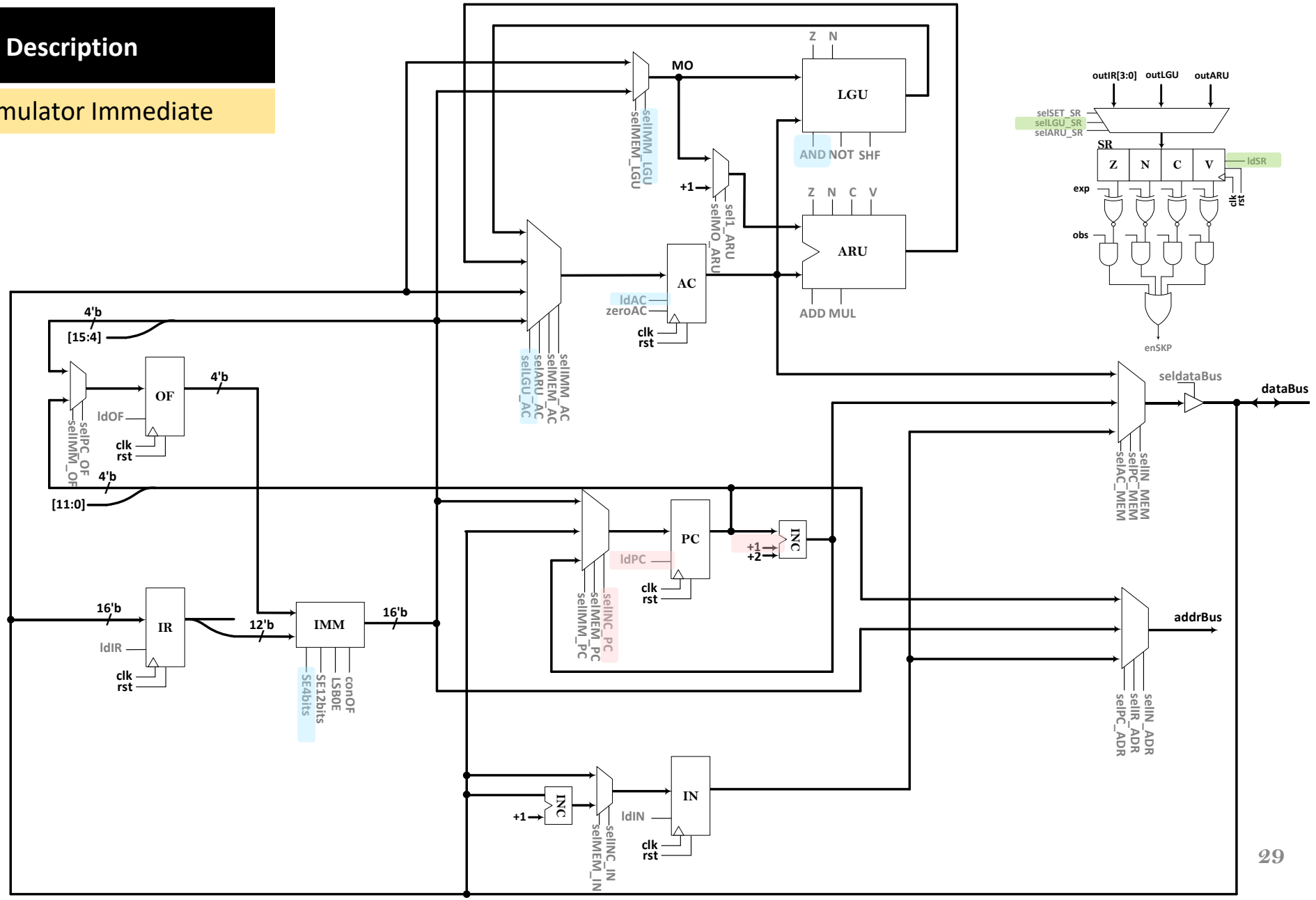
PUNEH as a Test-Case Processor

Opcode	Instruction	Description
0110	ANm	AND Accumulator Immediate

Exec1

ANm :

```
SE4bits = 1;
selIMM_LGU = 1;
AND = 1;
selLGU_AC = 1;
ldAC = 1;
selLGU_SR = 1;
ldSR = 4'b1100;
INC1 = 1;
selINC_PC = 1;
ldPC = 1;
```



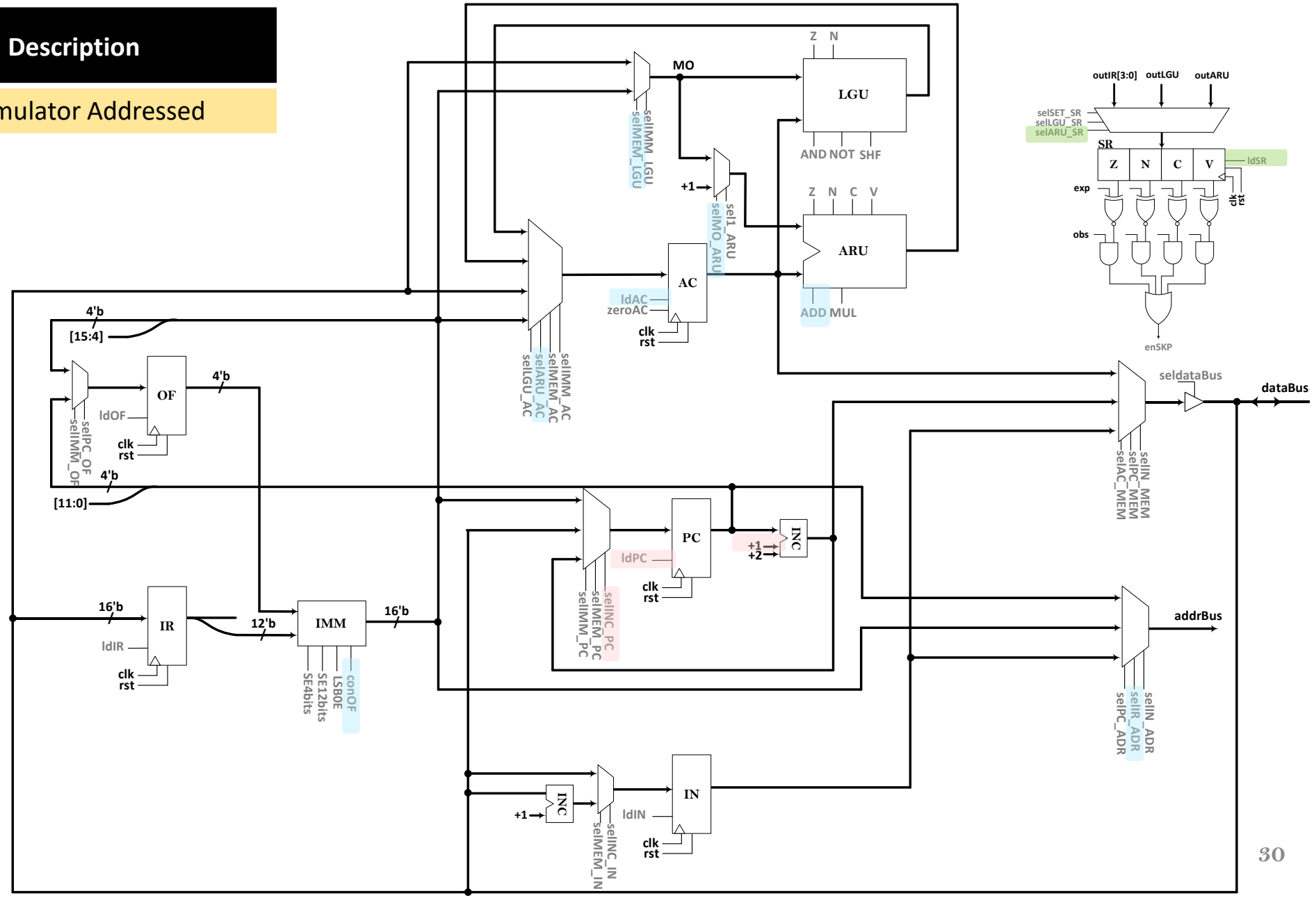
PUNEH as a Test-Case Processor

Opcode	Instruction	Description
1001	ADa	Add Accumulator Addressed

Exec1

ADa :

```
conOF = 1;
selIR_ADR = 1;
readMEM = 1;
selMEM_LGU = 1;
selMO_ARU  = 1;
ADD = 1;
selARU_AC = 1;
ldAC = 1;
selARU_SR = 1;
ldSR = 4'b1111;
INC1 = 1;
selINC_PC = 1;
ldPC = 1;
```



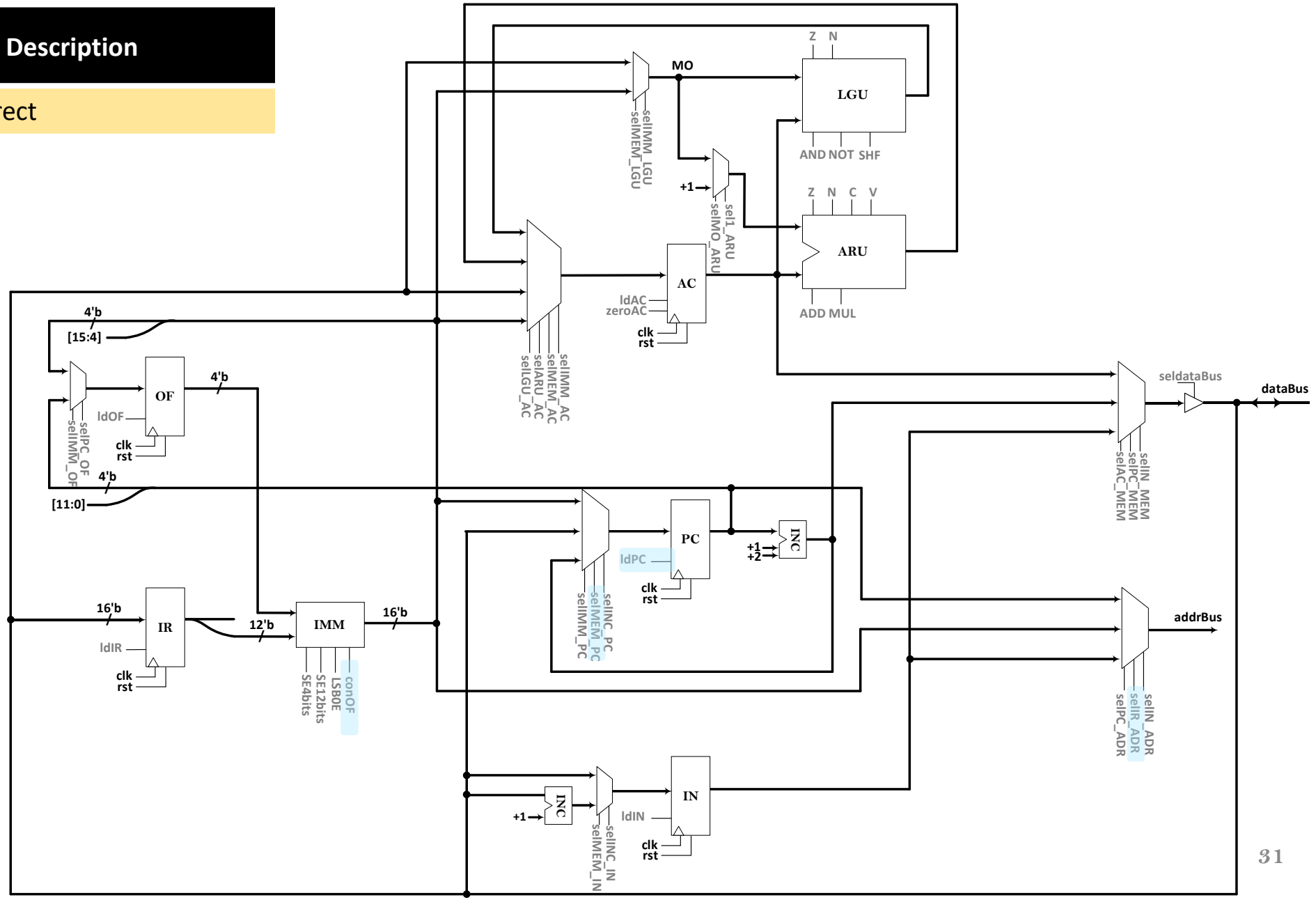
PUNEH as a Test-Case Processor

Opcode	Instruction	Description
1101	JMn	Jump Indirect

Exec1

JMn :

```
conOF = 1;
selIR_ADR = 1;
readMEM = 1;
selMEM_PC = 1;
ldPC = 1;
```

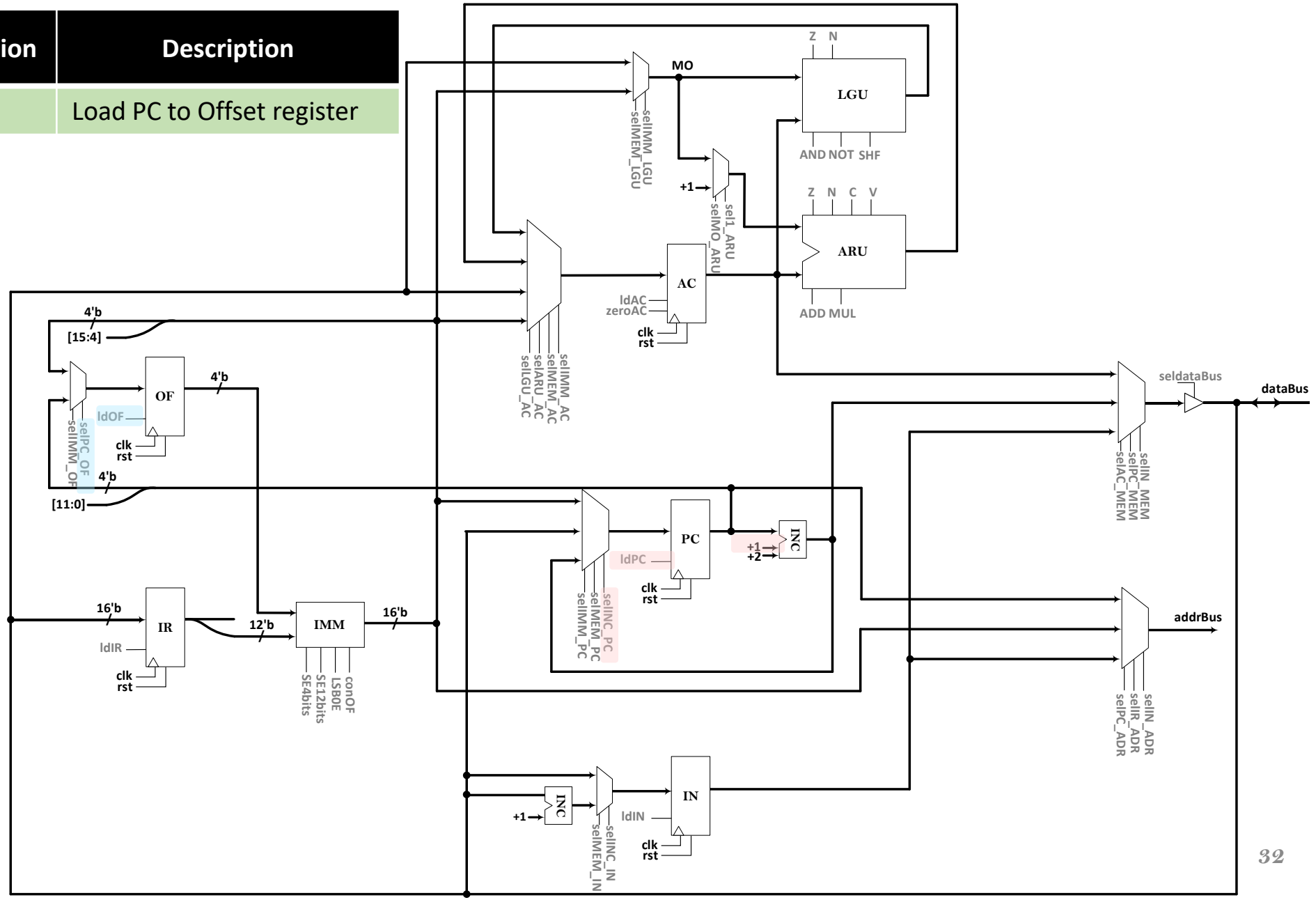


PUNEH as a Test-Case Processor

Opcode			Instruction	Description
1111	0000	00000000	LPO	Load PC to Offset register

Exec1

```
LPO  :
    selPC_OF = 1;
    ldOF  = 1;
    INC1 = 1;
    selINC_PC = 1;
    ldPC = 1;
```

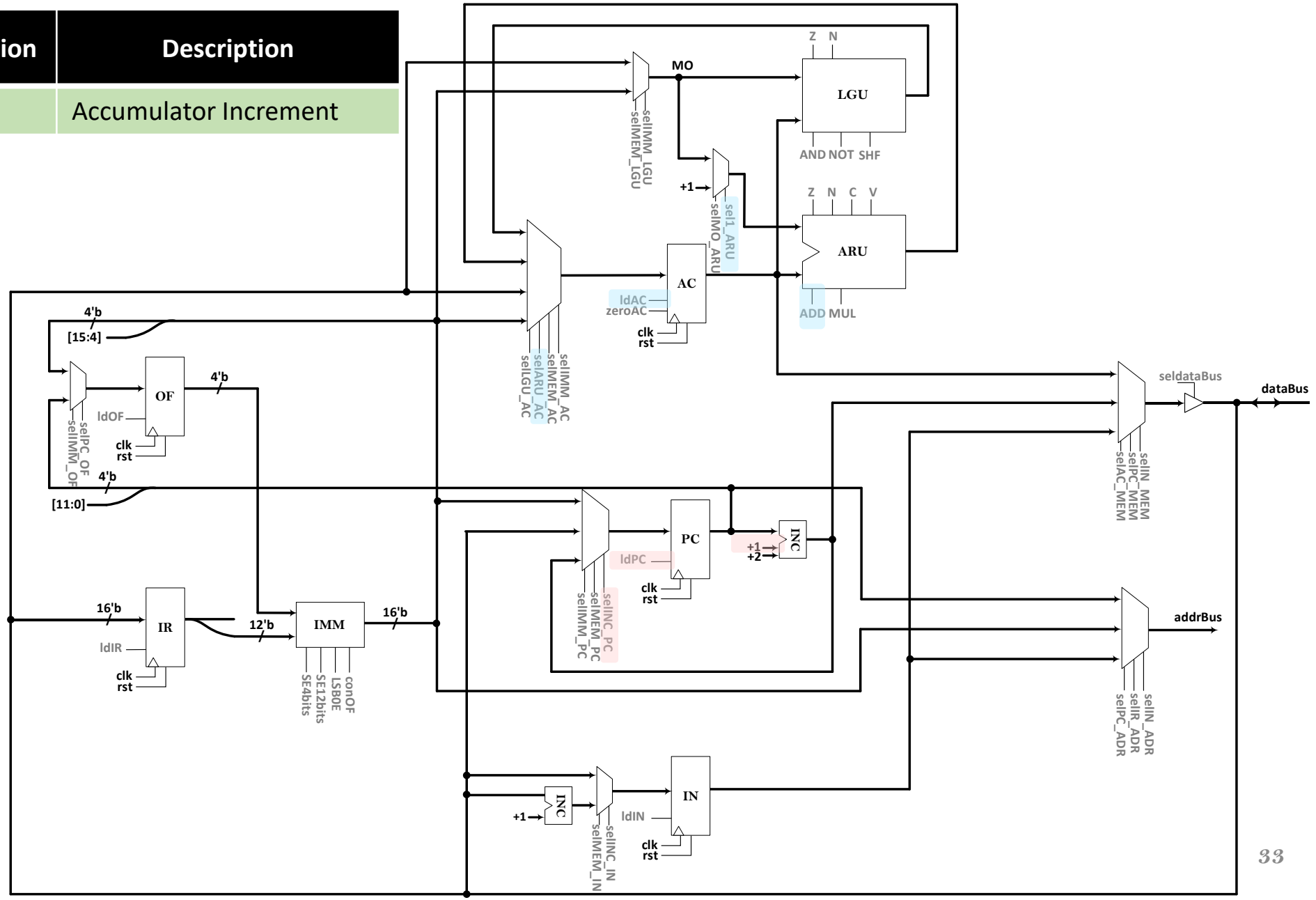


PUNEH as a Test-Case Processor

Opcode			Instruction	Description
1111	0000	00000100	ACI	Accumulator Increment

Exec1

```
ACI :
    sel1_ARU = 1;
    ADD = 1;
    selARU_AC = 1;
    ldAC = 1;
    INC1 = 1;
    selINC_PC = 1;
    ldPC = 1;
```

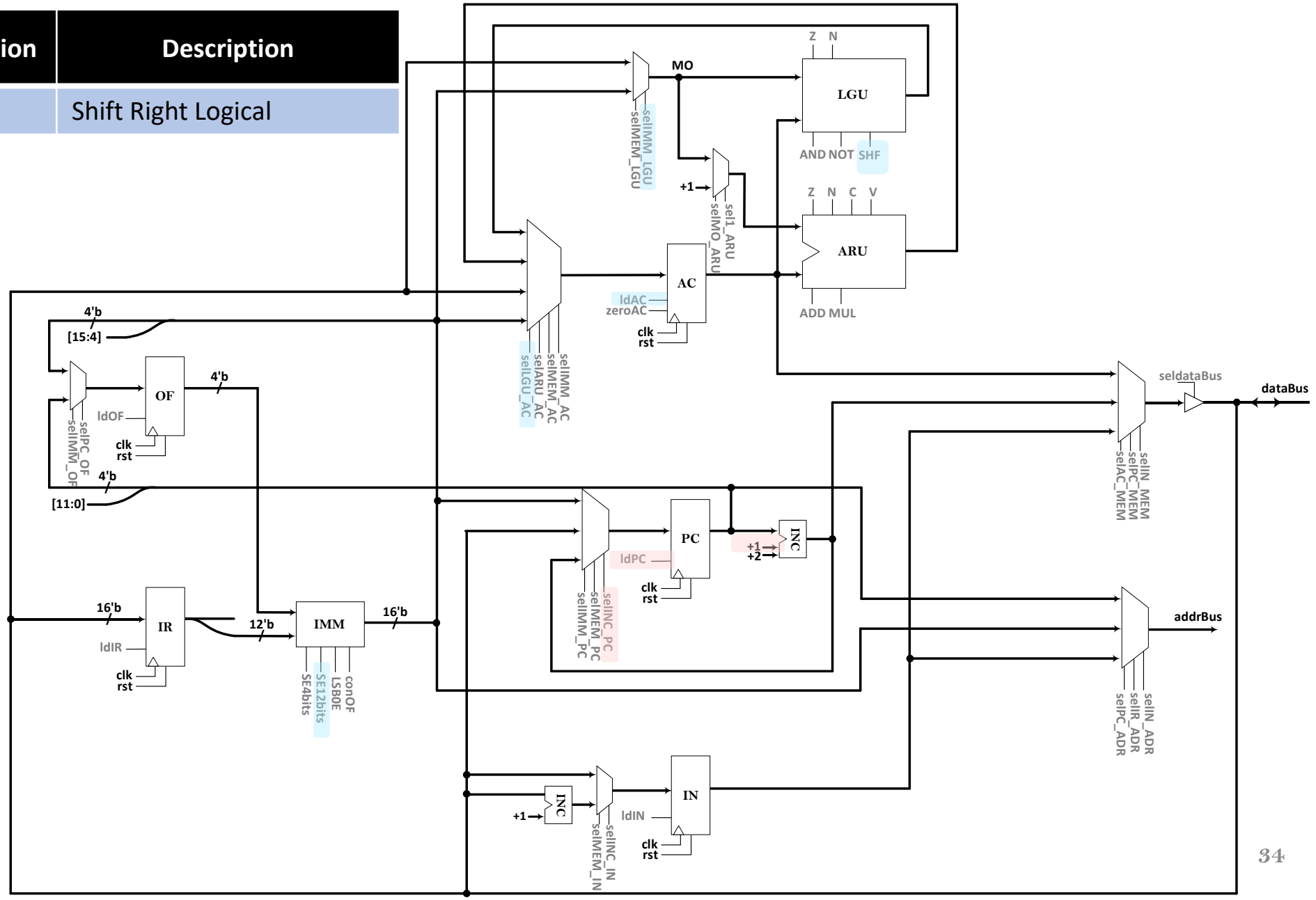


PUNEH as a Test-Case Processor

Opcode		Instruction	Description
1111	0011	SRL	Shift Right Logical

Exec1

```
SRL :
    SE12bits = 1;
    selIMM_LGU = 1;
    SHF = 2'b01;
    selLGU_AC = 1;
    ldAC = 1;
    INC1 = 1;
    selINC_PC = 1;
    ldPC = 1;
```

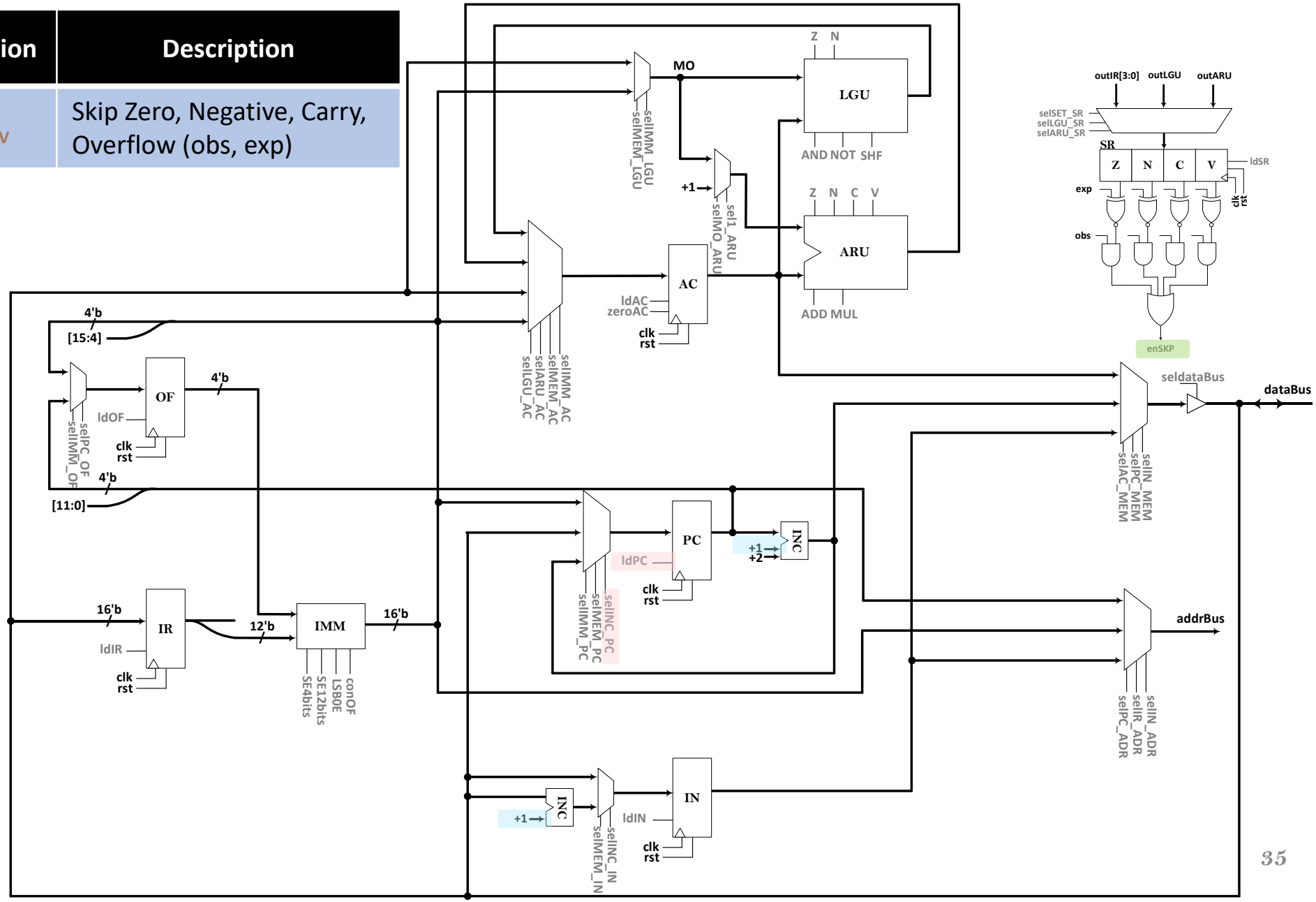


PUNEH as a Test-Case Processor

Opcode		Instruction	Description
1111	0101	$SKP_{Z,N,C,V}$	Skip Zero, Negative, Carry, Overflow (obs, exp)

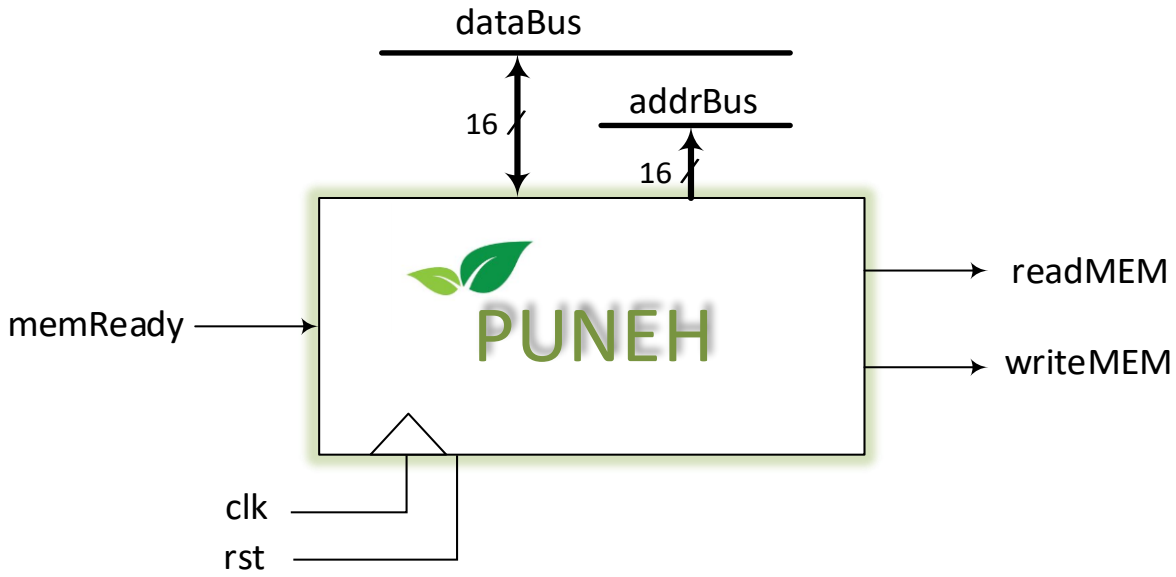
Exec1

```
SKP :
    if (enSKP)
        INC2 = 1;
    else
        INC1 = 1;
    selINC_PC = 1;
    ldPC = 1;
```



PUNEH as a Test-Case Processor

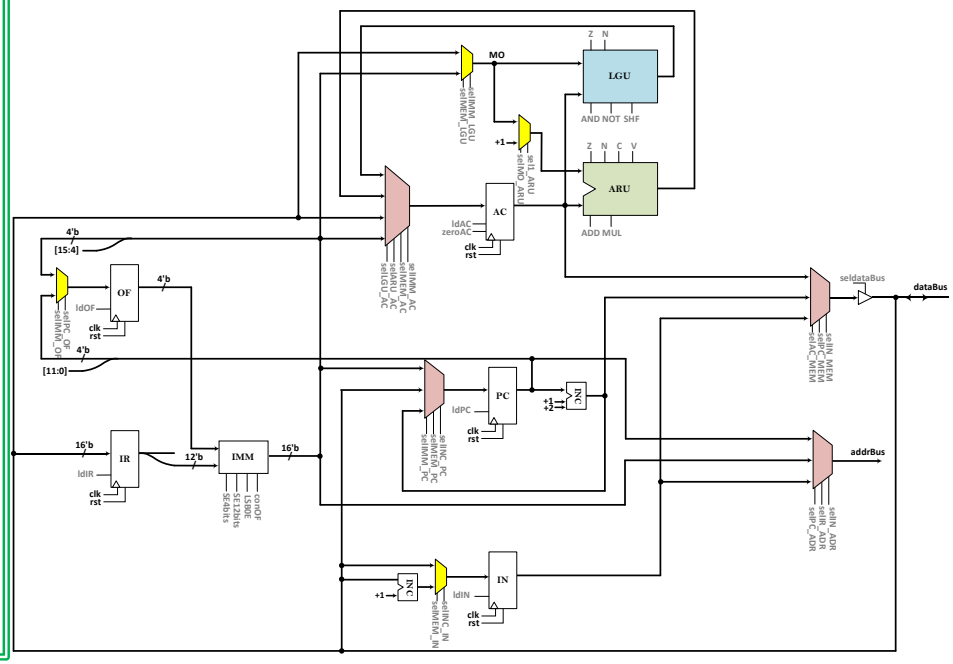
- **Multi-Cycle Implementation**
  - Datapath Design
  - Controller Design
  - **Datapath Description**
  - **Controller Description**
  - **The Complete Design**



PUNEH as a Test-Case Processor

- **Datapath**
- **LGU, ARU, Mux2to1, Mux4to1**

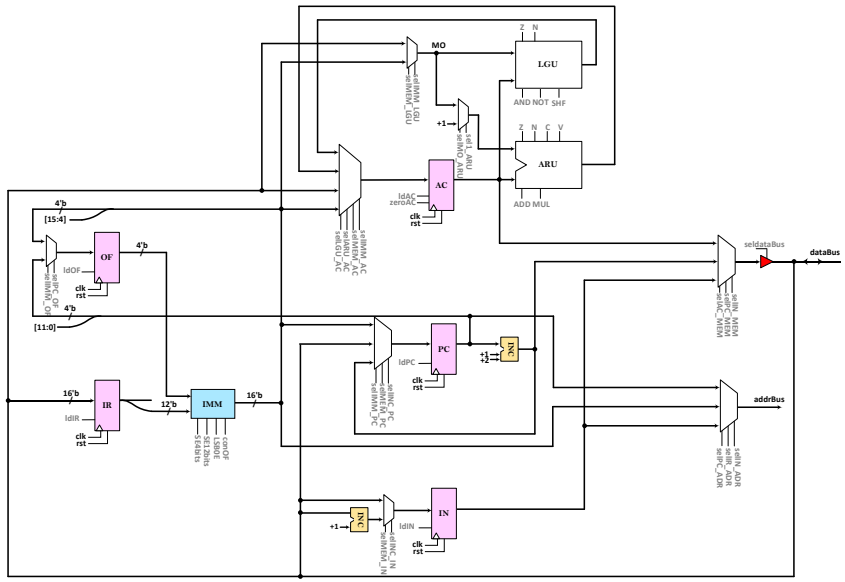
```
45
46      // Arithmetic Units
47      LGU LGU1 (AC_MEM, MO, LGU_AC, AND, NOT, SHF, Z1, N1);
48      ARU ARU1 (AC_MEM, MO_ARU, ARU_AC, ADD, MUL, Z2, N2, C, V);
49
50      // MUXes
51      Mux2to1 #(16) mux1  (16'd1, MO, sel1_ARU, selMO_ARU, MO_ARU);
52      Mux2to1 #(16) mux2  (IMM_AC, MEM_AC, selIMM_LGU, selMEM_LGU, MO);
53      Mux4to1 #(16) mux3  (IMM_AC, MEM_AC, ARU_AC, LGU_AC, selIMM_AC,
54                           selMEM_AC, selARU_AC, selLGU_AC, toAC);
55      Mux2to1 #(4)  mux4  (PC_MEM[15:12], IMM_AC[3:0], selPC_OF, selIMM_OF, toOF);
56      Mux4to1 #(16) mux5  (INC_PC, MEM_AC, IMM_AC, 16'bz, selINC_PC,
57                           selMEM_PC, selIMM_PC, 1'bz, toPC);
58      Mux2to1 #(16) mux6  (INC2_out, MEM_AC, selINC_IN, selMEM_IN, toIN);
59      Mux4to1 #(16) mux7  (IN_MEM, INC_PC, AC_MEM, 16'bz, selIN_MEM,
60                           selPC_MEM, selAC_MEM, 1'bz, toBuff);
61      Mux4to1 #(16) mux8  (IN_MEM, IMM_AC, PC_MEM, 16'bz, selIN_ADR,
62                           selIR_ADR, selPC_ADR, 1'bz, addrBus);
63      Mux2to1 #(2)  mux9  (2'd2, 2'd1, INC2, INC1, toINC);
64      Mux4to1 #(4)  mux10 (IRout[3:0], {Z1, N1, 2'bz}, {Z2, N2, C, V}, 4'bz, selSET_SR,
65                           selLGU_SR, selARU_SR, 1'bz, toSR);
66
```



PUNEH as a Test-Case Processor

- **Datapath**
- **Registers, IMM, INC, TriBuff**

```
67      // Registers
68      Register #(16) AC (clk, rst, toAC, AC_MEM, ldAC, zeroAC);
69      Register #(16) IR (clk, rst, MEM_AC, IRout, ldIR, 1'b0);
70      Register #(16) PC (clk, rst, toPC, PC_MEM, ldPC, 1'b0);
71      Register #(16) IN (clk, rst, toIN, IN_MEM, ldIN, 1'b0);
72      Register #(4)  OF (clk, rst, toOF, OFout, ldOF, 1'b0);
73      Register #(1)  SR_Z (clk, rst, toSR[3], w[3], ldSR[3], 1'b0);
74      Register #(1)  SR_N (clk, rst, toSR[2], w[2], ldSR[2], 1'b0);
75      Register #(1)  SR_C (clk, rst, toSR[1], w[1], ldSR[1], 1'b0);
76      Register #(1)  SR_V (clk, rst, toSR[0], w[0], ldSR[0], 1'b0);
77
78      // The Other Components
79      IMM IMM1 (IRout[11:0], OFout, IMM_AC, conOF, SE12bits, SE4bits, LSB0E);
80      INC INC_1 (PC_MEM, toINC, INC_PC);
81      INC INC_2 (MEM_AC, 2'd1, INC2_out);
82      Tristate TriBuff (toBuff, dataBus, seldataBus);
83
```

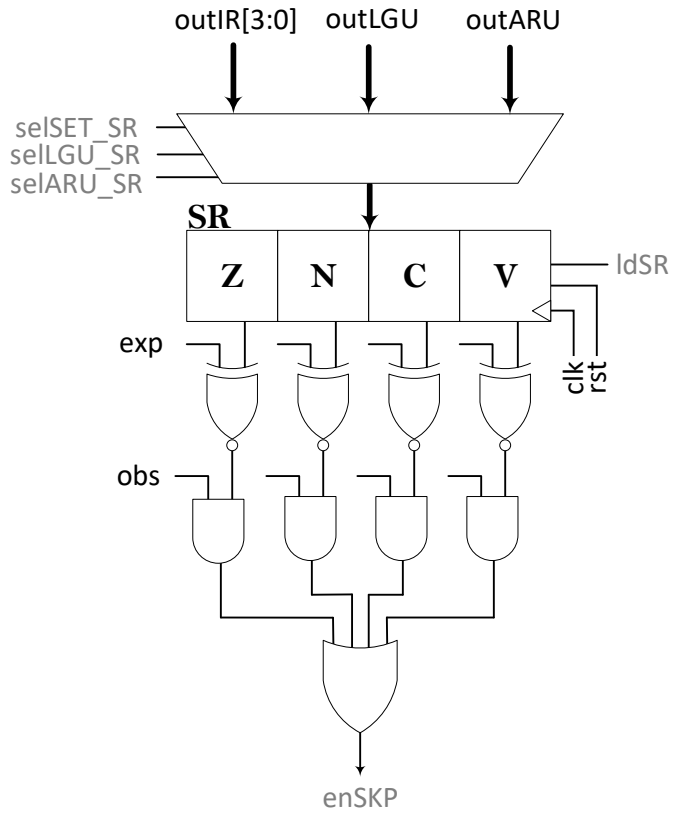


PUNEH as a Test-Case Processor

- Datapath
- Flag registers
- Flag assignments

```
73      Register #(1)  SR_Z (clk, rst, toSR[3], w[3], ldSR[3], 1'b0);
74      Register #(1)  SR_N (clk, rst, toSR[2], w[2], ldSR[2], 1'b0);
75      Register #(1)  SR_C (clk, rst, toSR[1], w[1], ldSR[1], 1'b0);
76      Register #(1)  SR_V (clk, rst, toSR[0], w[0], ldSR[0], 1'b0);
77
```

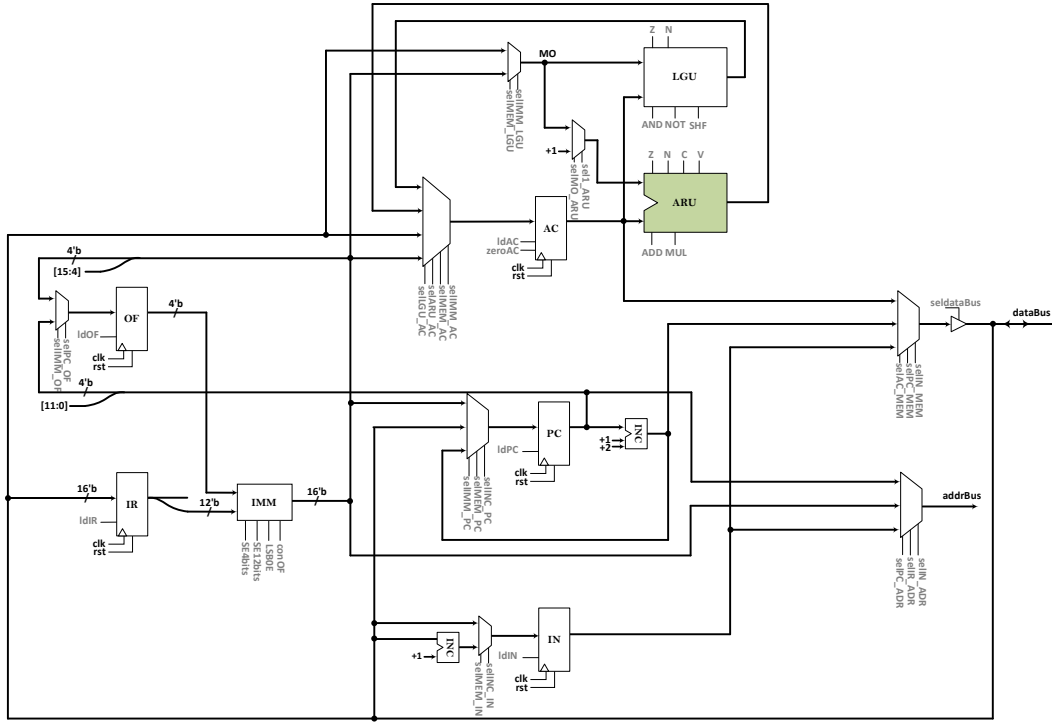
```
84      // Flags
85      assign exp = IRout[3:0];
86
87      assign w[4] = w[0] ~^ exp[0];
88      assign w[5] = w[1] ~^ exp[1];
89      assign w[6] = w[2] ~^ exp[2];
90      assign w[7] = w[3] ~^ exp[3];
91
92      assign obs = IRout[7:4];
93
94      assign w[8]  = w[4] & obs[0];
95      assign w[9]  = w[5] & obs[1];
96      assign w[10] = w[6] & obs[2];
97      assign w[11] = w[7] & obs[3];
98
99      assign enSKP = w[8] | w[9] | w[10] | w[11];
100
101     assign MEM_AC = dataBus;
102
103 endmodule
```



PUNEH as a Test-Case Processor

- **ARU (Arithmetic Unit)**

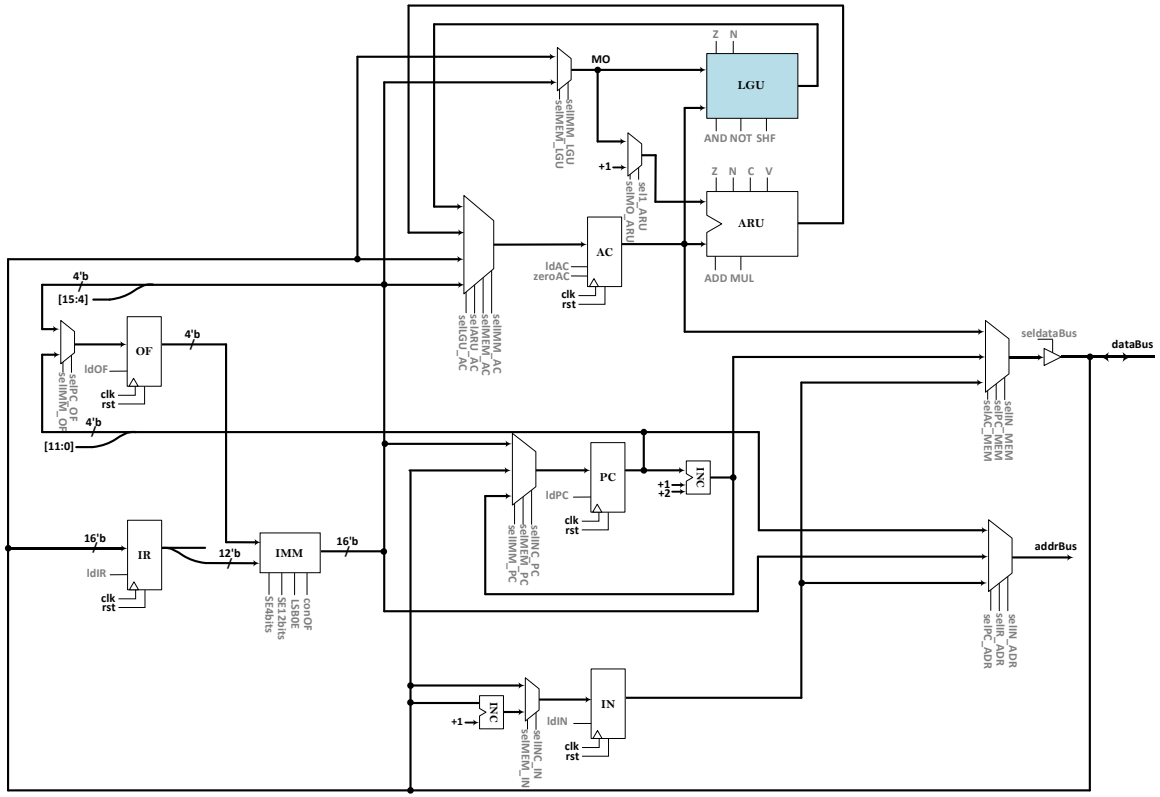
```
module ARU (

    input  logic signed [15:0] in0, in1,
    output logic signed [15:0] out,
    input  logic ADD, MUL,
    output logic Z, N, C, V
);
    always_comb begin
        C = 1'b0;
        out = 16'b0;

        if (ADD)
            {C, out} = in0 + in1;
        else if (MUL)
            out = in0[7:0] * in1[7:0];
    end

    assign Z = ~|out;
    assign N = out[15];
    assign V = (in0[15] & in1[15] & ~out[15]) || (~in0[15] & ~in1[15] & out[15]);

endmodule
```



PUNEH as a Test-Case Processor

- **LGU (Logic Unit)**

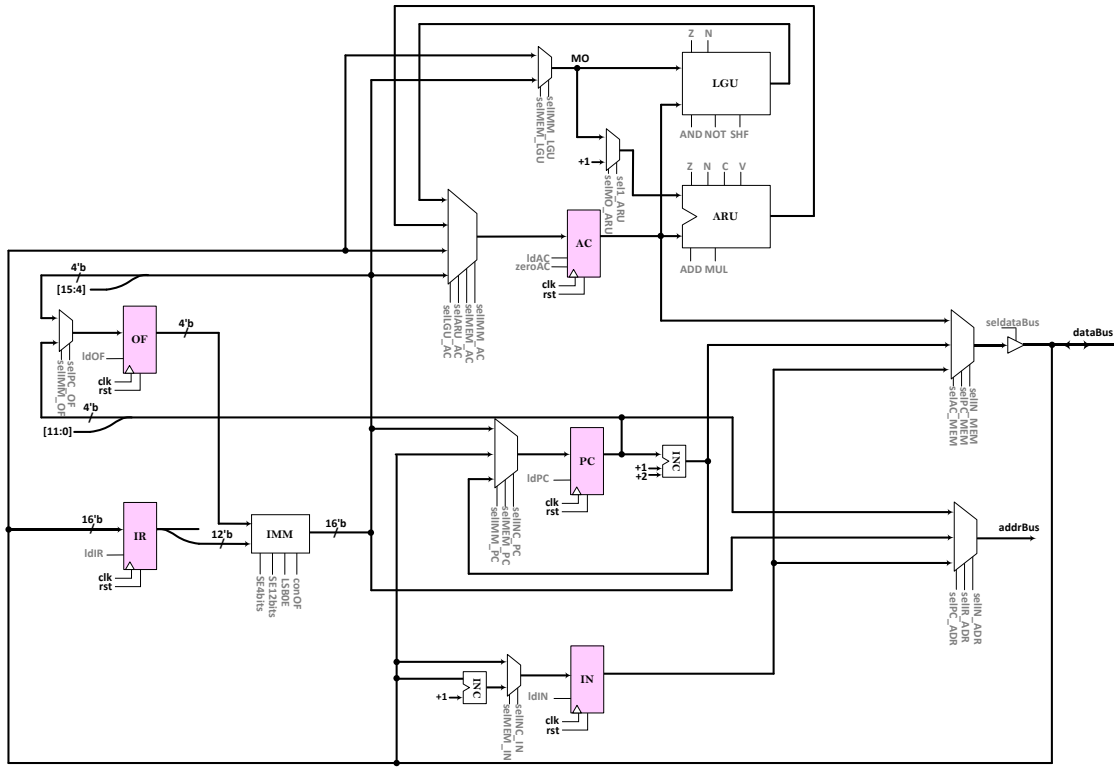
```
module LGU (
    input  logic signed [15:0] in0, in1,
    output logic signed [15:0] out,
    input  logic AND, NOT,
    input  logic [1:0] SHF,
    output logic Z, N
);
    always_comb begin
        out = 16'b0;

        if (AND == 1'b1)
            out = in0 & in1;
        else if (NOT == 1'b1)
            out = ~in0;
        else if (SHF == 2'b00)
            out = in0 >>> in1;
        else if (SHF == 2'b01)
            out = in0 >> in1;
        else if (SHF == 2'b10)
            out = in0 << in1;
    end

    assign Z = ~|out;
    assign N = out[15];

endmodule
```



PUNEH as a Test-Case Processor

- **Registers**

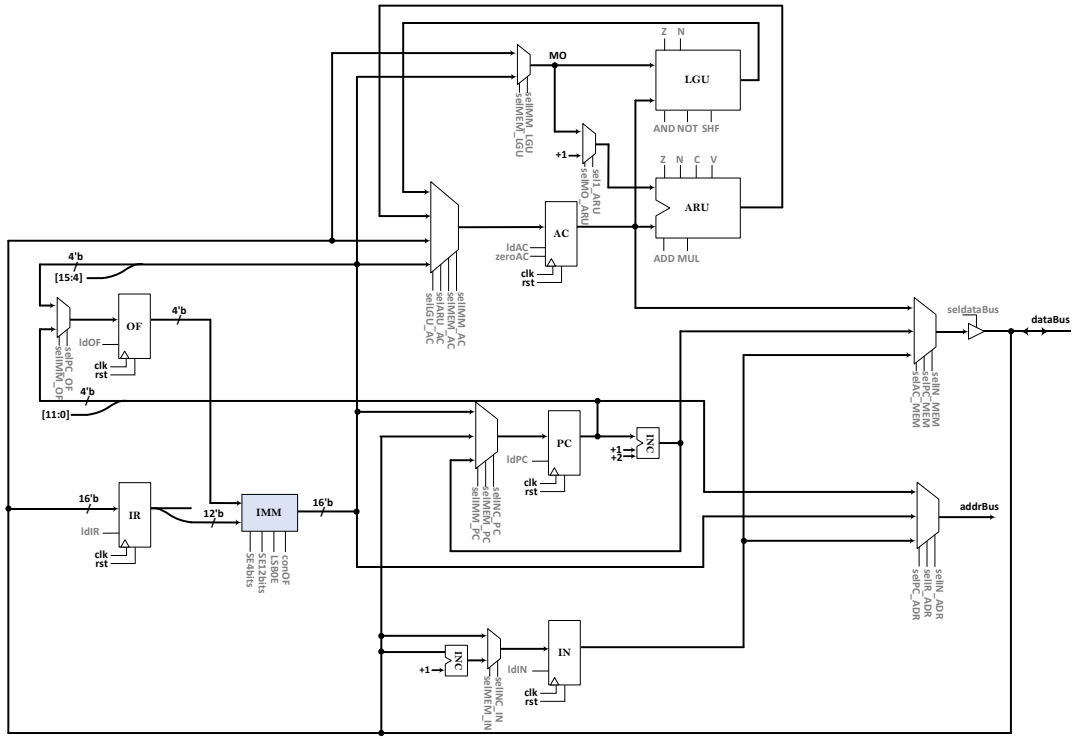
```
module Register #(parameter N) (
    input  logic clk, rst,
    input  logic [N-1:0] in,
    output logic [N-1:0] out,
    input  logic ld, clr
);
    always_ff @(posedge clk, posedge rst) begin
        if (rst)
            out <= 'b0;
        else if (clr)
            out <= 'b0;
        else if (ld)
            out <= in;
    end

endmodule
```



PUNEH as a Test-Case Processor

- **IMM (Immediate)**

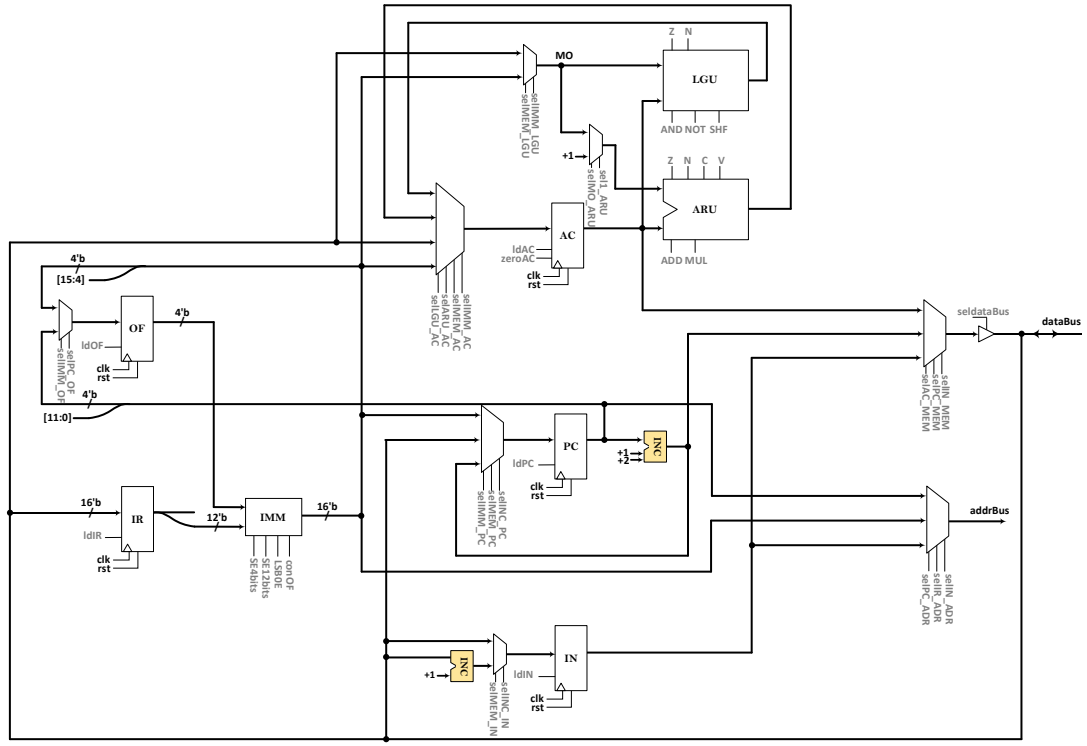
```
module IMM (
    input logic [11:0] in0,
    input logic [3:0]  in1,
    output logic [15:0] out,
    input logic conOF, SE12bits, SE4bits, LSB0E
);
    always_comb begin
        out = 16'b0;

        if (conOF)
            out = {in1, in0};
        else if(LSB0E)
            out = {in1,12'd0};
        else if (SE12bits)
            out = {{12{in0[3]}}, in0[3:0]};
        else if (SE4bits)
            out = {{4{in0[11]}}, in0};
    end

endmodule
```



PUNEH as a Test-Case Processor

- **INC (Incrementer)**

```
module INC (
    input  logic [15:0] in,
    input  logic [1:0]  inc_val,
    output logic [15:0] out
);
    assign out = in + inc_val;

endmodule
```



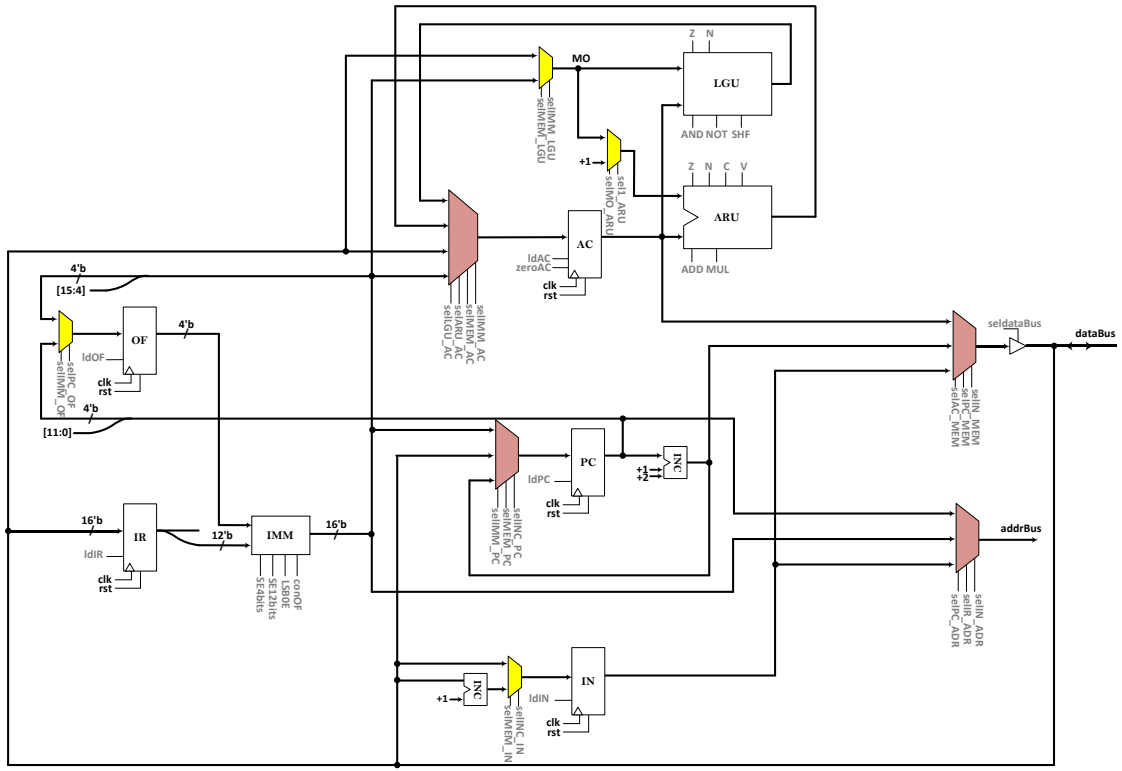
PUNEH as a Test-Case Processor

- Mux2to1

```
module Mux2to1 #(parameter N) (
    input logic [N-1:0] in0, in1,
    input logic sel0, sel1,
    output logic [N-1:0] out
);

    always_comb begin
        out = 16'b0;

        if (sel0)
            out = in0;
        else if (sel1)
            out = in1;
    end

endmodule
```

- Mux4to1

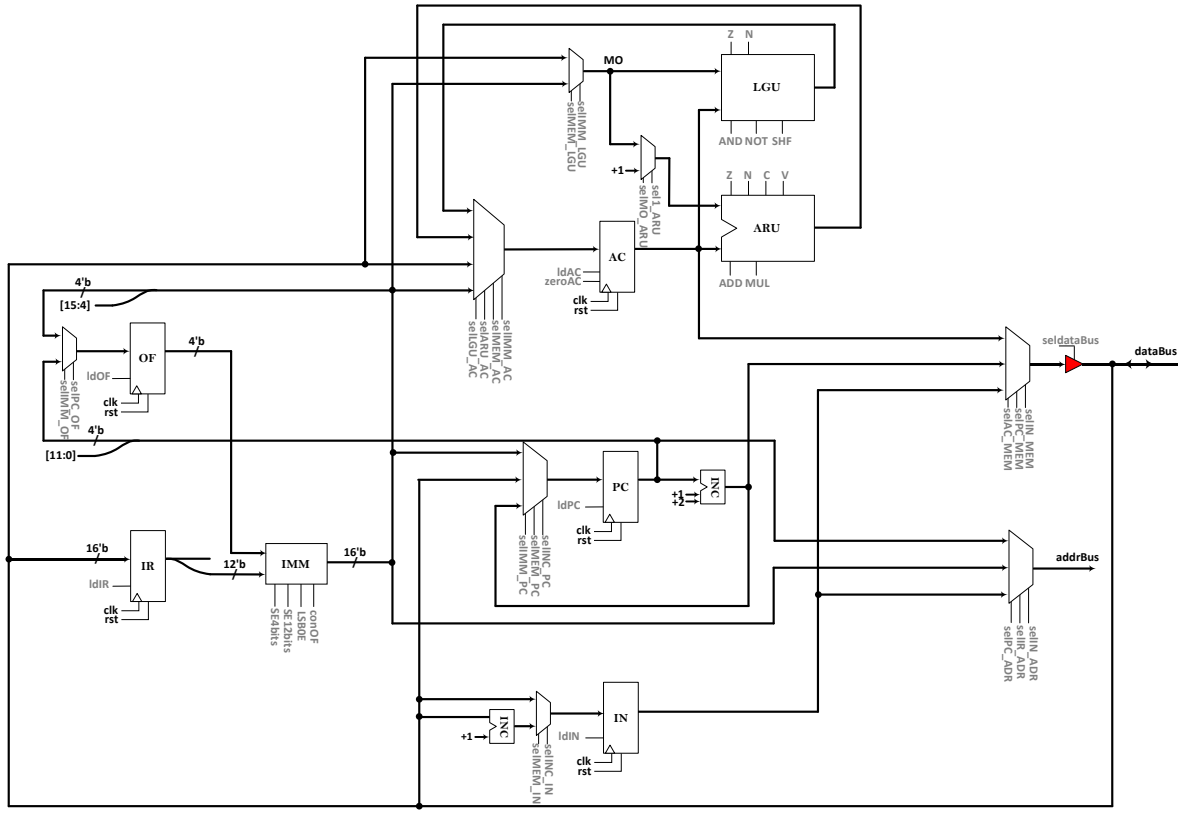
```
module Mux4to1 #(parameter N) (
    input  logic [N-1:0] in0, in1, in2, in3,
    input  logic sel0, sel1, sel2, sel3,
    output logic [N-1:0] out
);

    always_comb begin
        out = 16'b0;

        if (sel0)
            out = in0;
        else if (sel1)
            out = in1;
        else if (sel2)
            out = in2;
        else if (sel3)
            out = in3;
    end

endmodule
```



PUNEH as a Test-Case Processor

- **Tristate**

```
module Tristate (
    input  logic [15:0] in,
    output logic [15:0] out,
    input  logic oe
);
    assign out = oe ? in : 16'bz;

endmodule
```



PUNEH as a Test-Case Processor

- **Controller**

```
typedef enum logic [1:0] {
    fetch = 2'b00,
    exec1 = 2'b01,
    exec2 = 2'b10
} state_t;

state_t pstate, nstate;

typedef enum logic [3:0] {
    LDm     = 4'b0000,
    LDa     = 4'b0001,
    LDn     = 4'b0010,
    STa     = 4'b0011,
    STn     = 4'b0100,
    INa     = 4'b0101,
    ANm     = 4'b0110,
    ANa     = 4'b0111,
    ADm     = 4'b1000,
    ADa     = 4'b1001,
    ADn     = 4'b1010,
    MLa     = 4'b1011,
    JMa     = 4'b1100,
    JMn     = 4'b1101,
    JSR     = 4'b1110,
    INST15  = 4'b1111
} instr_type1;
```

```
typedef enum logic [3:0] {
    TYPE3   = 4'b0000,
    LOm     = 4'b0001,
    SRA     = 4'b0010,
    SRL     = 4'b0011,
    SLL     = 4'b0100,
    SKP     = 4'b0101,
    SET     = 4'b0110
} instr_type2;

typedef enum logic [7:0] {
    LPO     = 8'b00000000,
    LOP     = 8'b00000001,
    ACZ     = 8'b00000010,
    ACN     = 8'b00000011,
    ACI     = 8'b00000100
} instr_type3;
```

PUNEH as a Test-Case Processor

- **Huffman Model**

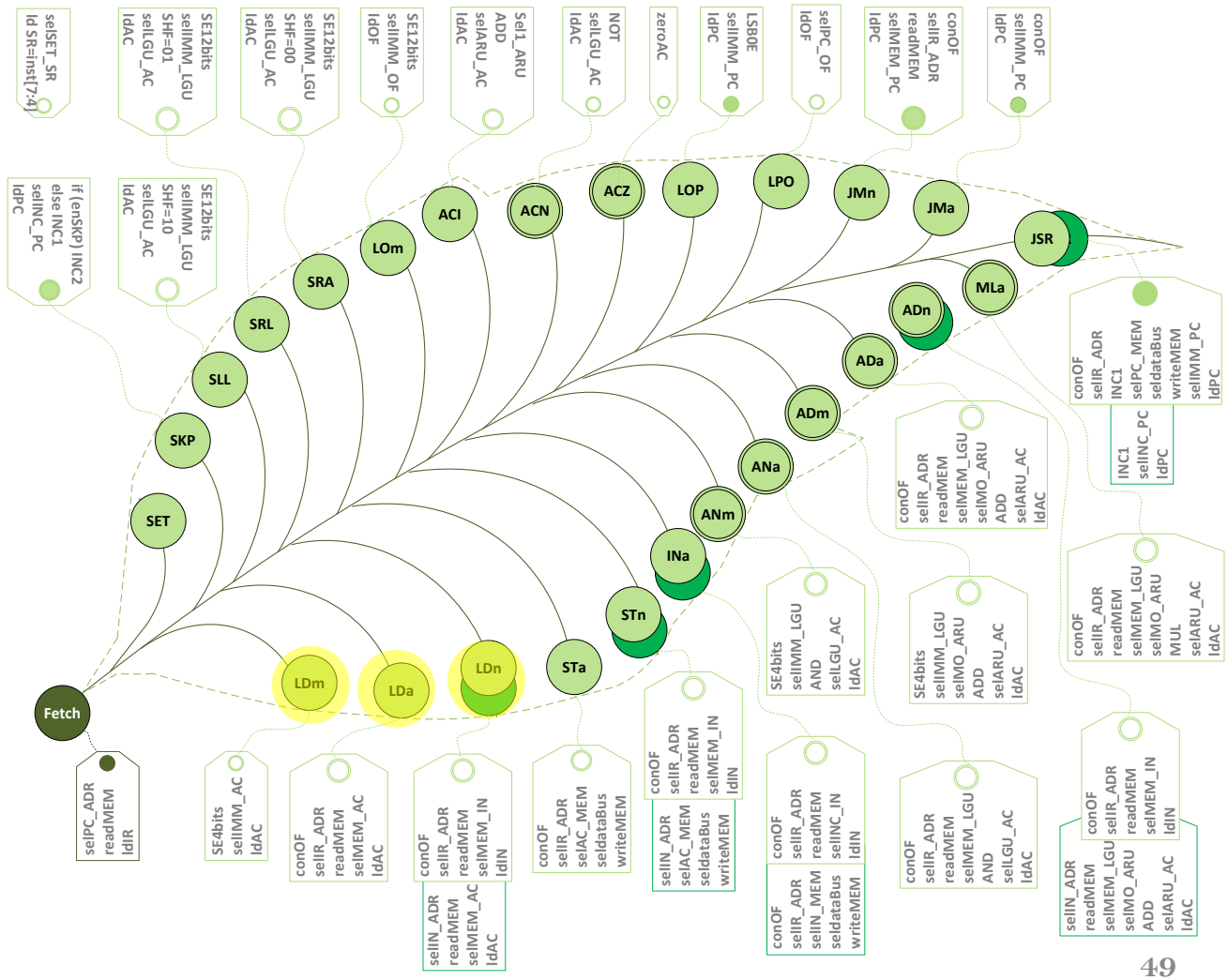
- **Register part of controller**

```
always_ff @(posedge clk, posedge rst) begin
    if (rst)
        pstate <= fetch;
    else
        pstate <= nstate;
end
```

PUNEH as a Test-Case Processor

- Output transition
- Fetch, Exec1

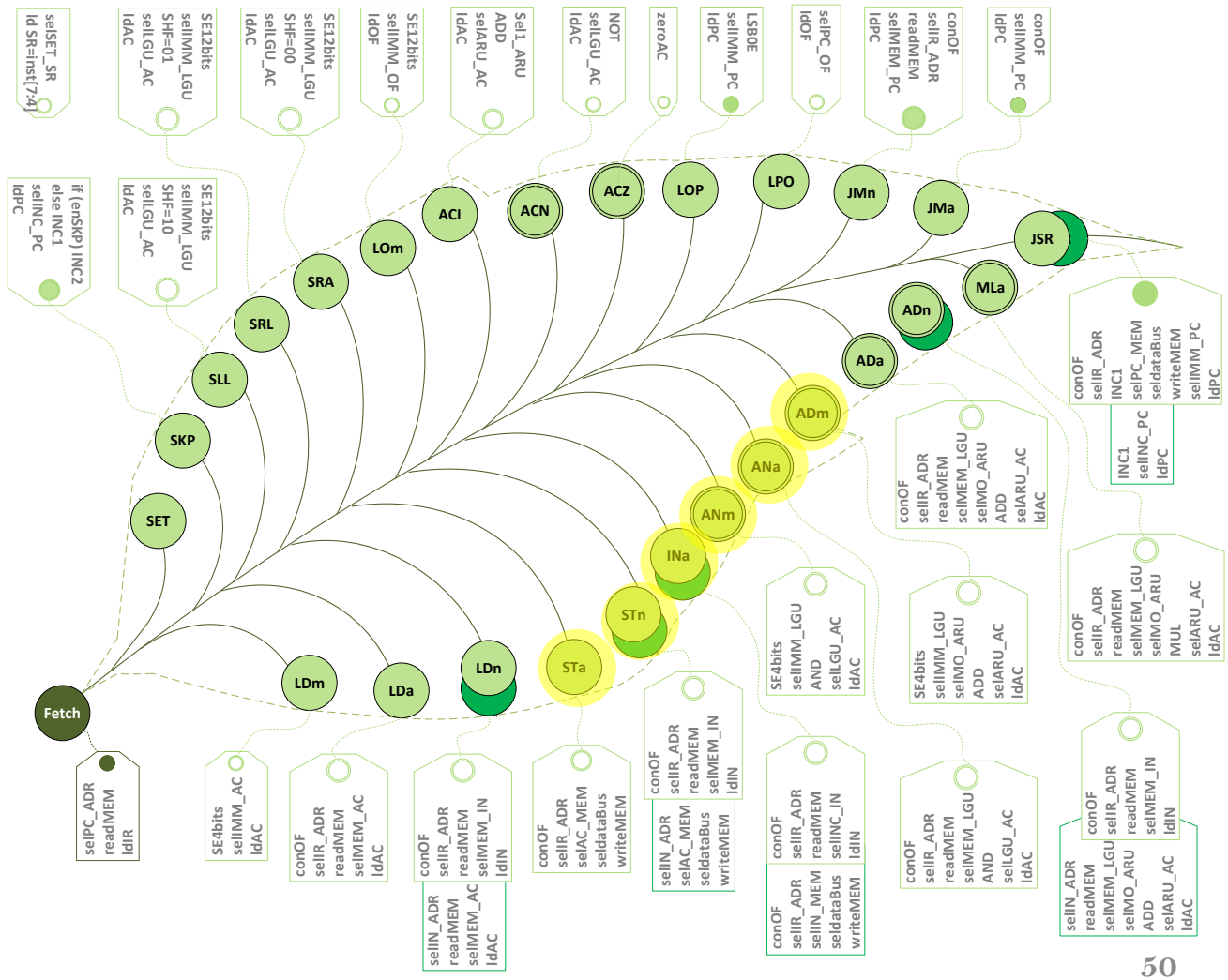
```
always_comb begin
    ldSR = 0; ldIR = 0; ldOF = 0; ldPC = 0;
    ldIN = 0; ldAC = 0; zeroAC = 0; seldataBus = 0;
    selPC_OF = 0; selIMM_OF = 0;  selINC_IN = 0; selMEM_IN = 0;
    selIMM_LGU = 0; selMEM_LGU = 0; sel1_ARU = 0; selMO_ARU = 0;
    selINC_PC = 0; selMEM_PC = 0; selIMM_PC = 0; selIN_ADR = 0;
    selIR_ADR = 0; selPC_ADR = 0;  selAC_MEM = 0;  selIMM_AC = 0;
    selMEM_AC = 0; selARU_AC = 0;  selLGU_AC = 0; conOF = 0;
    SE12bits = 0;  SE4bits = 0;  AND = 0;  NOT = 0;
    SHF = 0; ADD = 0;  MUL = 0;  readMEM = 0;
    writeMEM = 0; INC1 = 0;  INC2 = 0;
    selSET_SR = 0; selARU_SR = 0; selLGU_SR = 0;
    selIN_MEM=0; selPC_MEM=0; LSB0E = 0;

    case (pstate)
        fetch : begin
            selPC_ADR = 1;  ldIR = 1;  readMEM = 1;
        end
        exec1 : begin case (inst[15:12])
            LDm : begin
                SE4bits = 1;  selIMM_AC = 1;  ldAC = 1;
                INC1 = 1;  selINC_PC = 1;  ldPC = 1;
            end
            LDa : begin
                conOF = 1;  selIR_ADR = 1;  readMEM = 1;
                selMEM_AC = 1;  ldAC = 1;
                INC1 = 1;  selINC_PC = 1;  ldPC = 1;
            end
            LDn : begin
                conOF = 1;  selIR_ADR = 1;  readMEM = 1;
                selMEM_IN = 1;  ldIN = 1;
            end
```



PUNEH as a Test-Case Processor

- Output transition
- Exec1

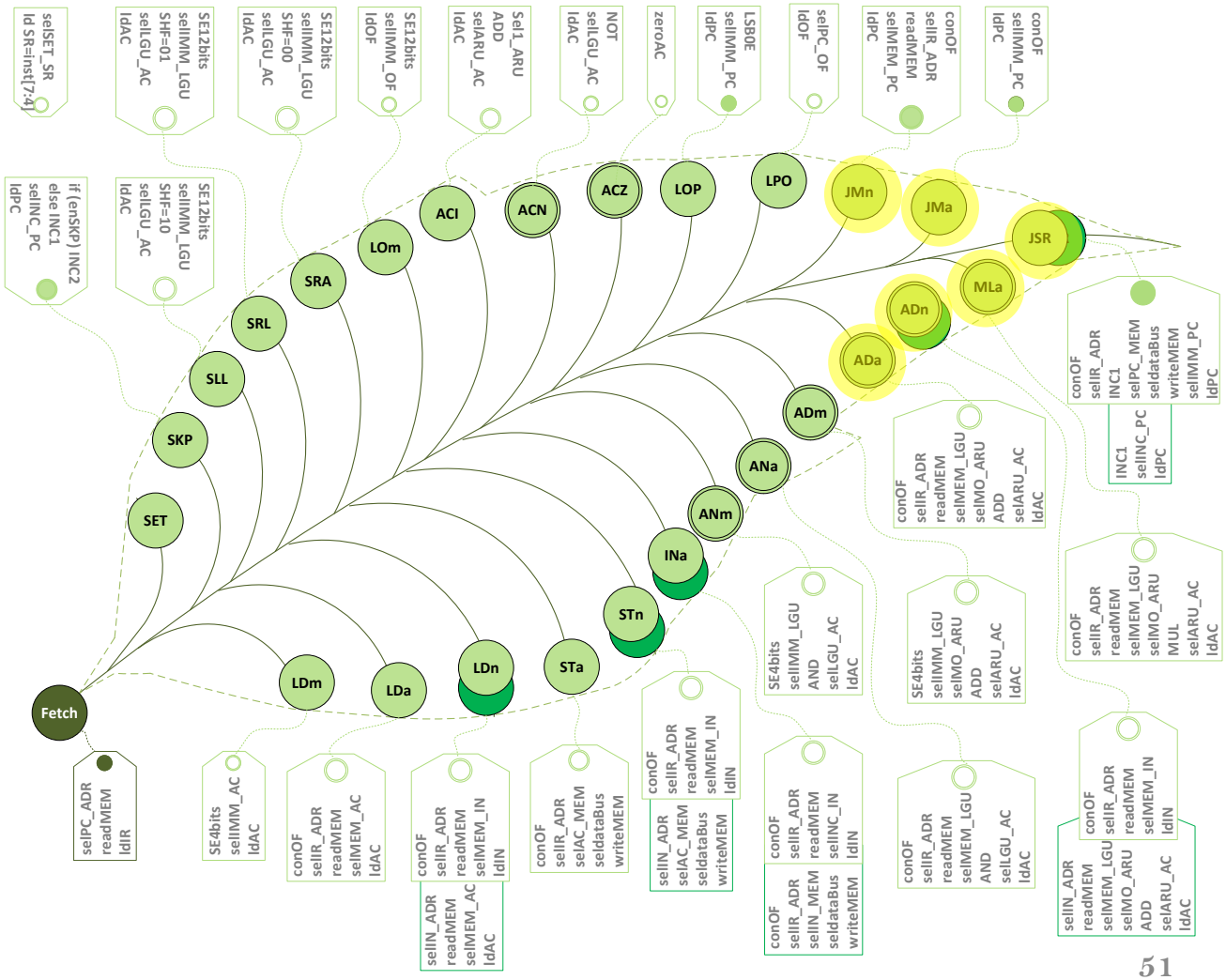
```
113             STa : begin
114                 conOF = 1;  selIR_ADR = 1;  selAC_MEM = 1;
115                 seldataBus = 1;  writeMEM = 1;
116                 INC1 = 1;  selINC_PC = 1;  ldPC = 1;
117             end
118             STn : begin
119                 conOF = 1;  selIR_ADR = 1;  readMEM = 1;
120                 selMEM_IN = 1;  ldIN = 1;
121             end
122             INa : begin
123                 conOF = 1;  selIR_ADR = 1;  readMEM = 1;
124                 selINC_IN = 1;  ldIN = 1;
125             end
126             ANm : begin
127                 SE4bits = 1;  selIMM_LGU = 1;  AND = 1;
128                 selLGU_AC = 1;  ldAC = 1;
129                 selLGU_SR = 1;  ldSR = 4'b1100;
130                 INC1 = 1;  selINC_PC = 1;  ldPC = 1;
131             end
132             ANa : begin
133                 conOF = 1;  selIR_ADR = 1;  readMEM = 1;
134                 selMEM_LGU = 1;  AND = 1;  selLGU_AC = 1; ldAC = 1;
135                 selLGU_SR = 1;  ldSR = 4'b1100;
136                 INC1 = 1;  selINC_PC = 1;  ldPC = 1;
137             end
138             ADm : begin
139                 SE4bits = 1;  selIMM_LGU = 1;  selMO_ARU = 1;
140                 ADD = 1;  selARU_AC = 1;  ldAC = 1;
141                 selARU_SR = 1;  ldSR = 4'b1111;
142                 INC1 = 1;  selINC_PC = 1;  ldPC = 1;
143             end
```



PUNEH as a Test-Case Processor

- Output transition
- Exec1

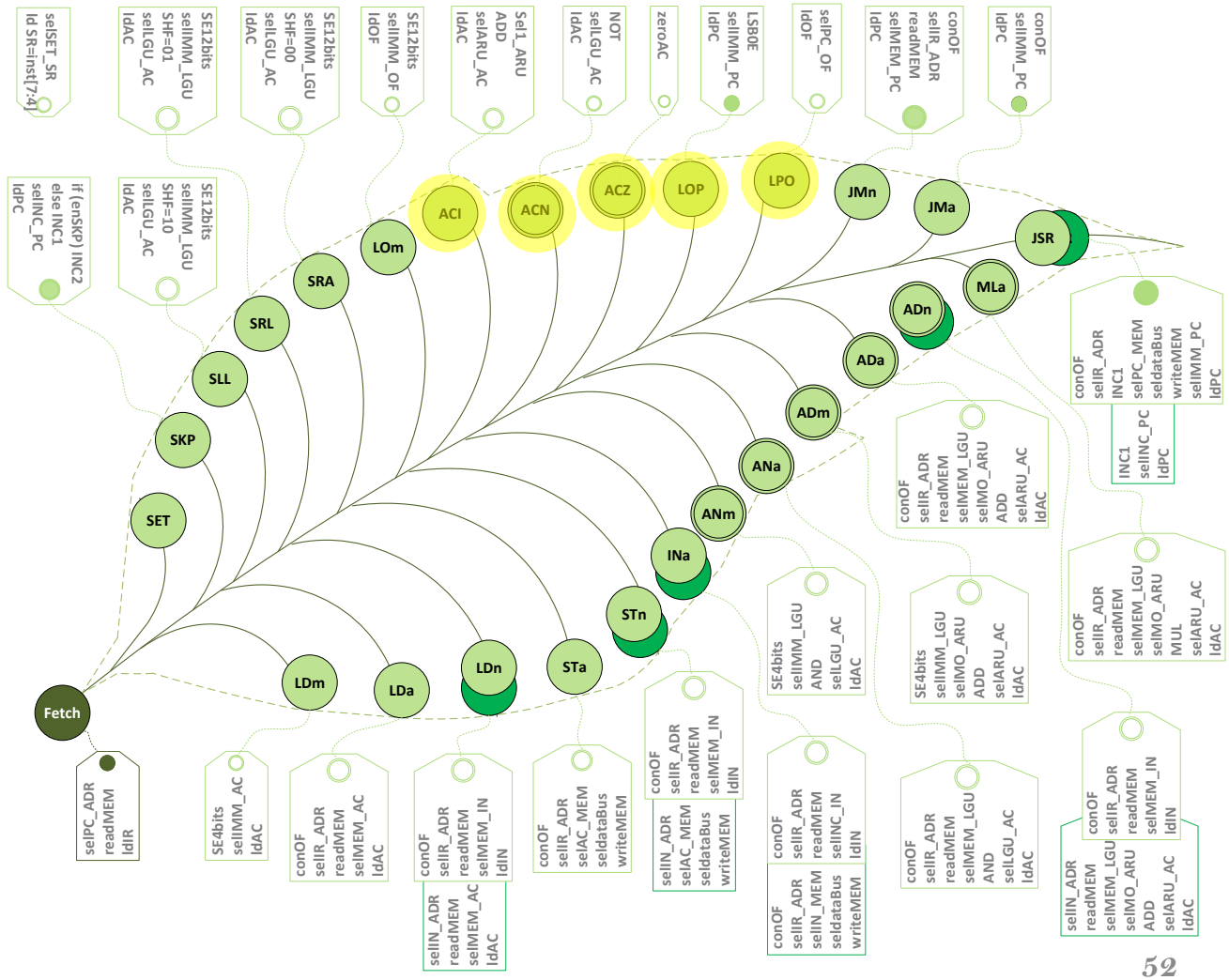
```
144                 ADa : begin
145                     conOF = 1;  selIR_ADR = 1;  readMEM = 1;
146                     selMEM_LGU = 1;  selMO_ARU  = 1;  ADD = 1;
147                     selARU_AC = 1;  ldAC = 1;
148                     selARU_SR = 1;  ldSR = 4'b1111;
149                     INC1 = 1;  selINC_PC = 1;  ldPC = 1;
150                 end
151                 ADn : begin
152                     conOF = 1;  selIR_ADR = 1;  readMEM = 1;
153                     selMEM_IN = 1;  ldIN = 1;
154                 end
155                 MLa : begin
156                     conOF = 1;  selIR_ADR = 1;  readMEM = 1;
157                     selMEM_LGU = 1;  selMO_ARU = 1;  MUL = 1;
158                     selARU_AC = 1;  ldAC = 1;
159                     selARU_SR = 1;  ldSR = 4'b1000;
160                     INC1 = 1;  selINC_PC = 1;  ldPC = 1;
161                 end
162                 JMa : begin
163                     conOF = 1;  selIMM_PC = 1;  ldPC = 1;
164                 end
165                 JMn : begin
166                     conOF = 1;  selIR_ADR = 1;  readMEM = 1;
167                     selMEM_PC = 1;  ldPC = 1;
168                 end
169                 JSR : begin
170                     conOF = 1;  selIR_ADR = 1;  INC1 = 1;
171                     selPC_MEM = 1;  seldataBus = 1;
172                     writeMEM = 1;  selIMM_PC = 1;  ldPC = 1;
173                 end
```



PUNEH as a Test-Case Processor

- Output transition
- Exec1

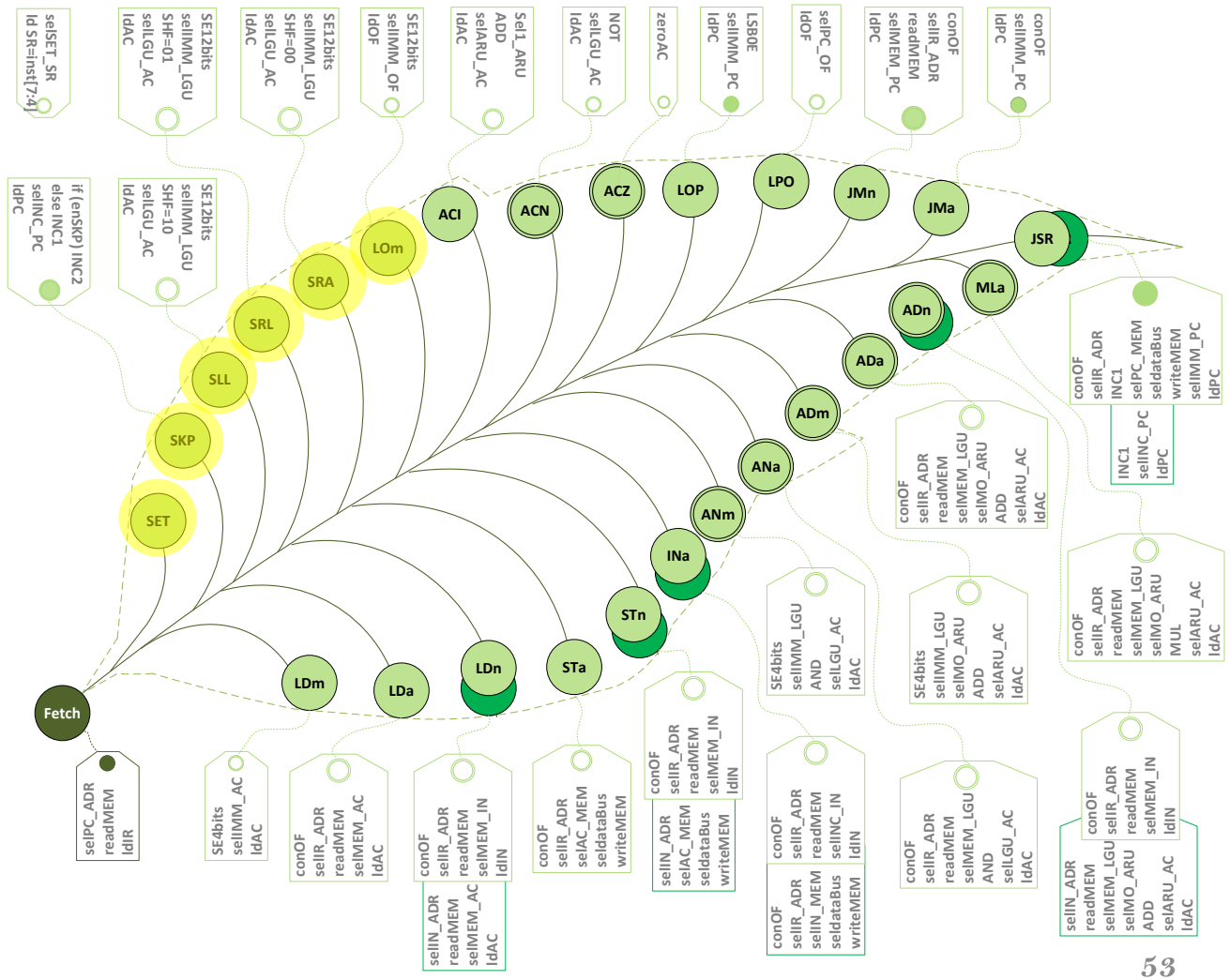
```
174                 INST15 : begin case (inst[11:8])
175                     TYPE1 : begin case (inst[7:0])
176                         LPO : begin
177                             selPC_OF = 1;  ldOF  = 1;
178                             INC1 = 1;  selINC_PC = 1;  ldPC = 1;
179                         end
180                         LOP : begin
181                             LSB0E = 1;  selIMM_PC = 1;  ldPC = 1;
182                         end
183                         ACZ : begin
184                             zeroAC = 1;
185                             selLGU_SR = 1;  ldSR = 4'b1100;
186                             INC1 = 1;  selINC_PC = 1;  ldPC = 1;
187                         end
188                         ACN : begin
189                             NOT = 1;  selLGU_AC = 1;  ldAC = 1;
190                             selLGU_SR = 1;  ldSR = 4'b1100;
191                             INC1 = 1;  selINC_PC = 1;  ldPC = 1;
192                         end
193                         ACI : begin
194                             sel1_ARU = 1;  ADD = 1;
195                             selARU_AC = 1;  ldAC = 1;
196                             INC1 = 1;  selINC_PC = 1;  ldPC = 1;
197                         end
198                         default : begin
199                             INC1 = 1;  selINC_PC = 1;  ldPC = 1;
200                         end
201                     endcase
202                     end
```



PUNEH as a Test-Case Processor

- Output transition
- Exec1

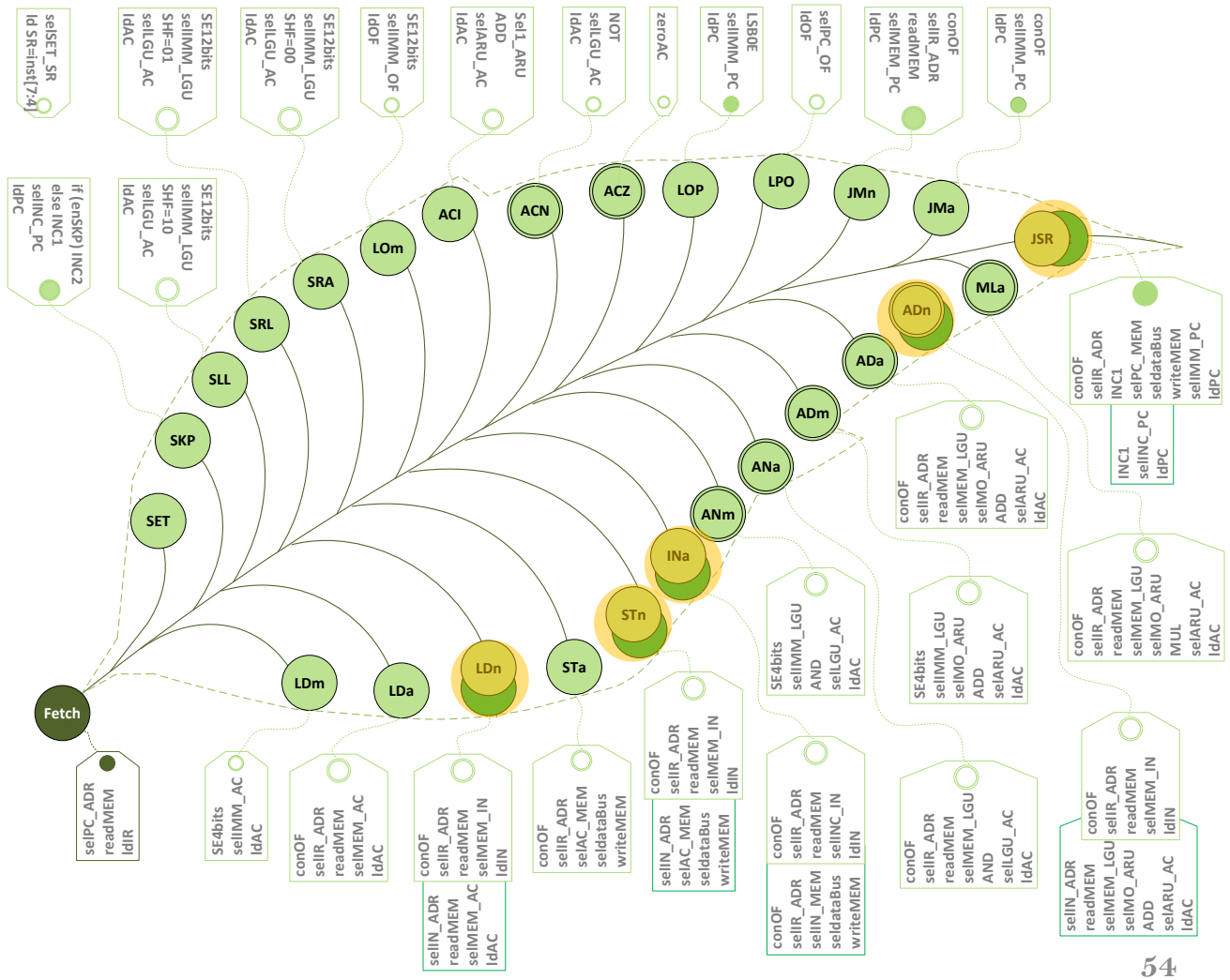
```
203                     LOm : begin
204                         SE12bits = 1;  selIMM_OF = 1;  ldOF = 1;
205                         INC1 = 1;  selINC_PC = 1;  ldPC = 1;
206                     end
207                     SRA : begin
208                         SE12bits = 1;  selIMM_LGU = 1;  SHF = 2'b00;
209                         selLGU_AC = 1;  ldAC = 1;
210                         INC1 = 1;  selINC_PC = 1;  ldPC = 1;
211                     end
212                     SRL : begin
213                         SE12bits = 1;  selIMM_LGU = 1;  SHF = 2'b01;
214                         selLGU_AC = 1;  ldAC = 1;
215                         INC1 = 1;  selINC_PC = 1;  ldPC = 1;
216                     end
217                     SLL : begin
218                         SE12bits = 1;  selIMM_LGU = 1;  SHF = 2'b10;
219                         selLGU_AC = 1;  ldAC = 1;
220                         INC1 = 1;  selINC_PC = 1;  ldPC = 1;
221                     end
222                     SKP : begin
223                         if (enSKP) INC2 = 1;
224                         else INC1 = 1;
225                         selINC_PC = 1;  ldPC = 1;
226                     end
227                     SET : begin
228                         selSET_SR = 1;  ldSR = inst[7:4];
229                         INC1 = 1;  selINC_PC = 1;  ldPC = 1;
230                     end
231                     default : begin
232                         INC1 = 1;  selINC_PC = 1;  ldPC = 1;
233                     end
234                 endcase
235                 end
236                     default : begin
237                         INC1 = 1;  selINC_PC = 1;  ldPC = 1;
238                     end
239             endcase
240             end
```



PUNEH as a Test-Case Processor

- Output transition
- Exec1, Exec2

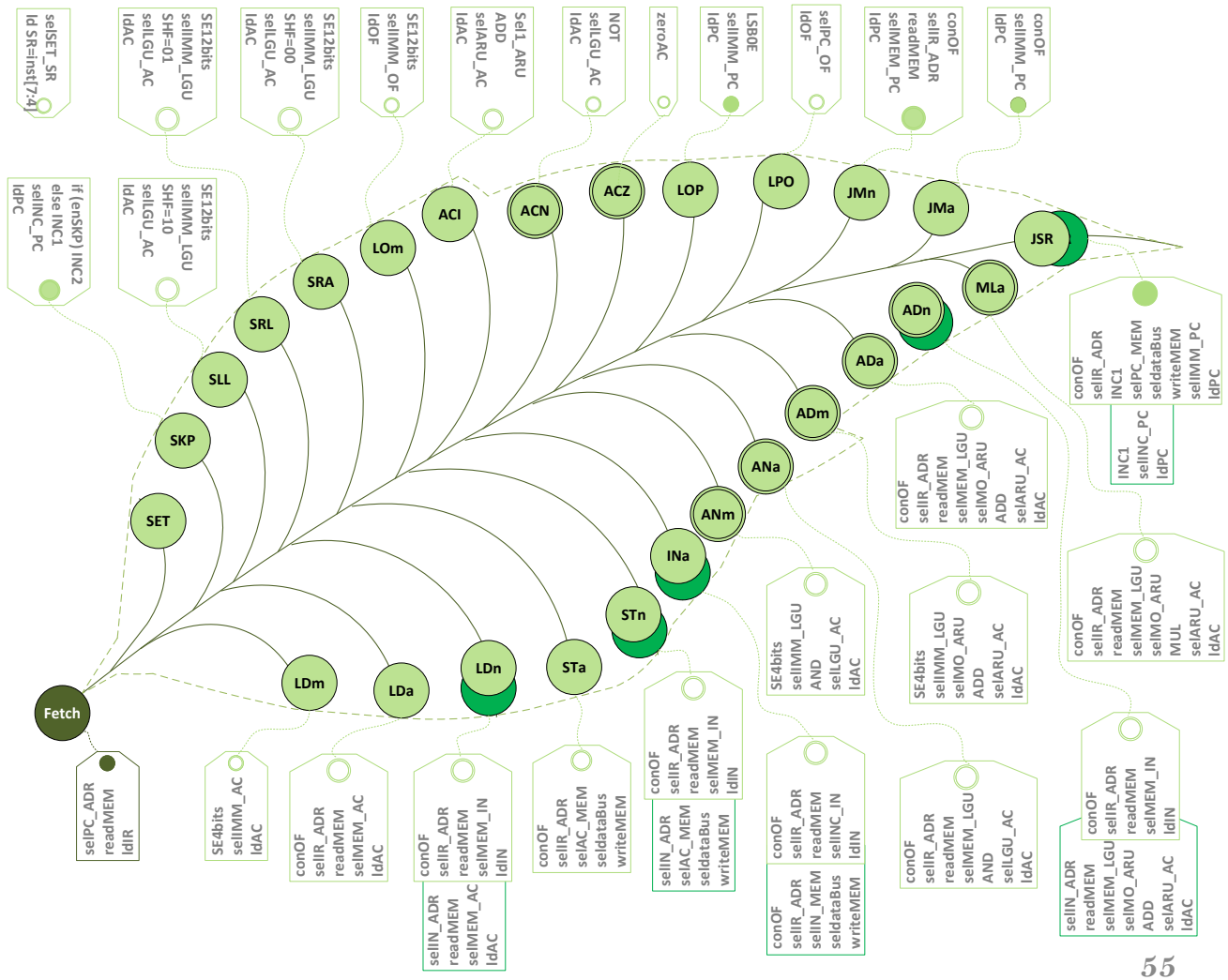
```
240                 end
241                 exec2 : begin case (inst[15:12])
242                     LDn : begin
243                         selIN_ADR = 1;  readMEM = 1;
244                         selMEM_AC = 1; ldAC = 1;
245                         INC1 = 1;  selINC_PC = 1;  ldPC = 1;
246                     end
247                     STn : begin
248                         selIN_ADR = 1;  selAC_MEM = 1;
249                         seldataBus = 1;  writeMEM = 1;
250                         INC1 = 1;  selINC_PC = 1;  ldPC = 1;
251                     end
252                     INa : begin
253                         conOF = 1;  selIR_ADR = 1;  selIN_MEM = 1;
254                         seldataBus = 1;  writeMEM = 1;
255                         INC1 = 1;  selINC_PC = 1;  ldPC = 1;
256                     end
257                     ADn : begin
258                         selIN_ADR = 1;  readMEM = 1;  selMEM_LGU = 1;
259                         selMO_ARU = 1;  ADD = 1;  selARU_AC = 1;  ldAC = 1;
260                         selARU_SR = 1;  ldSR = 4'b1111;
261                         INC1 = 1;  selINC_PC = 1;  ldPC = 1;
262                     end
263                     JSR : begin
264                         INC1 = 1;  selINC_PC = 1;  ldPC = 1;
265                     end
266                 endcase
267                 end
268             endcase
269         end
```



PUNEH as a Test-Case Processor

- **State transition**

```
always_comb begin
    nstate = fetch;
    case (pstate)
        fetch :
            nstate = exec1;
        exec1 : begin case (inst[15:12])
            LDm :    nstate = fetch;
            LDa :    nstate = fetch;
            LDn :    nstate = exec2;
            STa :    nstate = fetch;
            STn :    nstate = exec2;
            INa :    nstate = exec2;
            ANm :    nstate = fetch;
            ANa :    nstate = fetch;
            ADm :    nstate = fetch;
            ADa :    nstate = fetch;
            ADn :    nstate = exec2;
            MLa :    nstate = fetch;
            JMa :    nstate = fetch;
            JMn :    nstate = fetch;
            JSR :    nstate = exec2;
            INST15 : nstate = fetch;
            default: nstate = fetch;
        endcase
    end
        exec2 : begin
            nstate = fetch ;
        end
        default :
            nstate = fetch;
    endcase
end
```



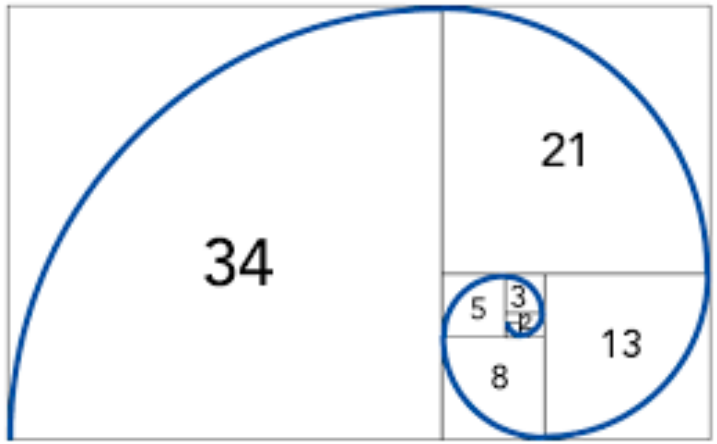
PUNEH as a Test-Case Processor

- Top Module
- Datapath
- Controller

```
module TopLevel (
    input  logic clk, rst,
    inout  logic [15:0] dataBus,
    output logic writeMEM, readMEM,
    output logic [15:0] addrBus
);
    logic [15:0] inst;
    logic [3:0]  ldSR;
    logic [1:0]  SHF;
    logic enSKP, ldIR, ldOF, ldPC, ldIN, ldAC, zeroAC, seldataBus, selPC_OF,
          selIMM_OF, selMEM_IN, selIMM_LGU, selMEM_LGU, sel1_ARU, selMO_ARU,
          selINC_PC, selMEM_PC, selIMM_PC, selIN_ADR, selIR_ADR, selPC_ADR,
          selAC_MEM, selIMM_AC, selMEM_AC, selARU_AC, selLGU_AC, conOF,
          SE12bits, SE4bits, AND, NOT, ADD, MUL,selSET_SR, selINC_IN,
          INC1, INC2, selARU_SR, selLGU_SR,selIN_MEM, selPC_MEM, LSB0E;

    Datapath DP (clk, rst, AND, NOT, ADD, MUL, sel1_ARU, selMO_ARU, selIMM_LGU,
                           selMEM_LGU, ldAC, zeroAC, selIMM_AC, selMEM_AC,  selARU_AC,
                           selLGU_AC, ldPC, selINC_PC, selMEM_PC, selIMM_PC, ldIN,
                           selINC_IN, selMEM_IN, conOF, SE12bits, SE4bits, ldIR, ldOF,
                           selPC_OF, selIMM_OF, selIN_ADR, selPC_ADR, selAC_MEM, selIR_ADR,
                           seldataBus, INC1, INC2, selSET_SR, selLGU_SR, selARU_SR,selIN_MEM, selPC_MEM, LSB0E,
                           dataBus, SHF, ldSR, addrBus, enSKP);

    Controller CU (clk, rst, enSKP, inst, ldIR, ldOF, ldPC, ldIN, ldAC, zeroAC, seldataBus,
                   selPC_OF, selIMM_OF, selINC_IN, selMEM_IN, selIMM_LGU, selMEM_LGU,
                   sel1_ARU, selMO_ARU, selINC_PC, selMEM_PC, selIMM_PC, selIN_ADR,
                   selIR_ADR, selPC_ADR, selAC_MEM, selIMM_AC, selMEM_AC, selARU_AC,
                   selLGU_AC, conOF, SE12bits, SE4bits, AND, NOT, ADD, MUL,
                   readMEM, writeMEM, selSET_SR, INC1, INC2, selARU_SR, selLGU_SR,selIN_MEM, selPC_MEM, LSB0E,
                   SHF, ldSR);

endmodule
```

PUNEH as a Test-Case Processor

- **Programming in PUNEH**
  - **Instruction-by-instruction testing**
  - **Fibonacci sequence**
    - **A series of numbers where each number is the sum of the two preceding ones**
    - **Starting with 0 and 1, resulting in 0, 1, 1, 2, 3, 5, 8, 13, …**
    - **F(n) = F(n-1) + F(n-2) for n > 1, with F(0) = 0 and F(1) = 1**



PUNEH as a Test-Case Processor

- Programming in PUNEH
- Fibonacci
  - C program

```c
#include<stdio.h>
int main()
{
 int n1=0,n2=1,n3,i,number;
 printf("Enter the number of ele
 scanf("%d",&number);
 printf("\n%d %d",n1,n2);
 for(i=2;i<number;++i)
 {
  n3=n1+n2;
  printf(" %d",n3);
  n1=n2;
  n2=n3;
 }
  return 0;
 }
```

F(10)=0,1,1,2,3, 5, 8, 13, 21, 34

  - Assembely Code

```
//*****************************************************************************/
//  File content description:
//  Fibonacci assembly code for the PUNEH processor
/*****************************************************************************/

    #define n 10
    #define a0 111
    #define a1 112
    #define t 113
    #define cntr 114
    #define pointer 115

    mem[0] = FA(LDm, -n+2);
    mem[1] = FA(STa, cntr);
    mem[2] = FA(LDm, 128);          // results will store in mem[128] to mem[128+abs(n)]
    mem[3] = FA(STa, pointer);
    mem[4] = FA(LDm, 0);
    mem[5] = FA(STa, a0);
    mem[6] = FA(STn, pointer);
    mem[7] = FA(INa, pointer);
    mem[8] = FA(LDm, 1);
    mem[9] = FA(STa, a1);
    mem[10] = FA(STn, pointer);
    mem[11] = FA(INa, pointer);
    // loop:
    mem[12] = FA(LDa, a0);
    mem[13] = FA(ADa, a1);
    mem[14] = FA(STa, t);
    mem[15] = FA(STn, pointer);
    mem[17] = FA(LDa, a1);
    mem[18] = FA(STa, a0);
    mem[19] = FA(LDa, t);
    mem[20] = FA(STa, a1);
    mem[21] = FA(INa, pointer);
    mem[22] = FA(INa, cntr);
    mem[23] = FA(LDa, cntr);
    mem[24] = FA(ADm, 0);
    mem[25] = NA(SKP, 0X8, 0X8);
    mem[26] = FA(JMa, 12);
```

  - Binary Code

```
//*****************************************************************************/
//  File content description:
//  Fibonacci binary code for the PUNEH processor
/*****************************************************************************/

0000111111111000
0011000001110010
0000000010000000
0011000001110011
0000000000000000
0011000001101111
0100000001110011
0101000001110011
0000000000000001
0011000001110000
0100000001110011
0101000001110011
0001000001101111
1001000001110000
0011000001110001
0100000001110011
0001000001110000
0011000001101111
0001000001110001
0011000001110000
0101000001110011
0101000001110010
0001000001110010
1000000000000000
1111010110001000
1100000000001100
```

PUNEH as a Test-Case Processor

- **Simulation Result**